

Workforce Platform Engineering Blueprint v2.0

2026-08-22

Workforce Platform - Engineering Blueprint v2.0

0. How to Use This Document

0.1 Source-of-truth hierarchy

0.2 Required module handoff package

Part I - Product Direction and Non-Negotiable Decisions

1. Product Definition

2. Product-First Commercial Rule

3. Locked Technology Baseline

4. Architectural Stance

5. What Is Expensive Abstraction vs Cheap Guardrail

Part II - Deployment and Infrastructure

6. Supported Deployment Profiles

7. On-Premise Reference Topology

8. Deployment Sizing Profiles

9. Public Careers Portal Boundary

10. AI Deployment Modes

Part III - Codebase and Multi-Developer Collaboration

11. Recommended Repository Structure

12. Dependency Rules

13. Parallel Workstream Model

14. Git and Pull Request Rules

15. Contract-First Workflow for Developers

Part IV - Database Architecture and Relationship Rules

16. PostgreSQL Schema Ownership

17. Database Rules

18. Cross-Module Foreign Keys and Ownership

19. Snapshot vs Reference Rule

20. Core Workforce ERD

21. Migration Ownership

Part V - API, Events, Permissions and Shared Contracts

22. API Conventions

23. Domain Events vs Integration Events

24. Permission Naming

25. Shared Identifiers and Value Objects

Part VI - Module Contracts

26. Tenancy & Legal Entities Module Contract

Owns

Explicitly does not own

Public commands

Public queries

Events published

Events/contracts consumed

Primary permissions

Owned tables

Non-negotiable invariants

Module Definition of Done

27. Identity & Access Module Contract

Owns

Explicitly does not own

Public commands

Public queries

Events published

Events/contracts consumed

Primary permissions

Owned tables

Non-negotiable invariants

Module Definition of Done

28. Organization Module Contract

Owns

Explicitly does not own

Public commands

Public queries

Events published

Events/contracts consumed

Primary permissions

Owned tables

Non-negotiable invariants

Module Definition of Done

29. People Module Contract

Owns

Explicitly does not own

Public commands

Public queries

Events published

Events/contracts consumed

Primary permissions

Owned tables

Non-negotiable invariants

Module Definition of Done

30. Documents & Contracts Module Contract

Owns

Explicitly does not own

Public commands

Public queries

Events published

Events/contracts consumed

Primary permissions

Owned tables

Non-negotiable invariants

Module Definition of Done

31. Attendance & Time Module Contract

Owns

Explicitly does not own

Public commands

Public queries

Events published

Events/contracts consumed

Primary permissions

Owned tables

Non-negotiable invariants

Module Definition of Done

32. Leave Module Contract

Owns

Explicitly does not own

Public commands

Public queries

Events published

Events/contracts consumed

Primary permissions

Owned tables

Non-negotiable invariants

Module Definition of Done

33. Approvals Module Contract

Owns

Explicitly does not own

Public commands

Public queries

Events published

Events/contracts consumed

Primary permissions

Owned tables

Non-negotiable invariants

Module Definition of Done

34. Payroll Module Contract

Owns

Explicitly does not own

Public commands

Public queries

Events published

Events/contracts consumed

Primary permissions

Owned tables

Non-negotiable invariants

Module Definition of Done

35. Egypt Compliance Module Contract

Owns

Explicitly does not own

Public commands

Public queries

Events published

Events/contracts consumed

Primary permissions

Owned tables

Non-negotiable invariants

Module Definition of Done

36. Termination & Settlement Module Contract

Owns

Explicitly does not own

Public commands

Public queries

Events published

Events/contracts consumed

Primary permissions

Owned tables

Non-negotiable invariants

Module Definition of Done

37. Recruitment / ATS Module Contract

Owns

Explicitly does not own

Public commands

Public queries

Events published

Events/contracts consumed

Primary permissions

Owned tables

Non-negotiable invariants

Module Definition of Done

38. Reporting & Analytics Module Contract

Owns

Explicitly does not own

Public commands

Public queries

Events published

Events/contracts consumed

Primary permissions

Owned tables

Non-negotiable invariants

Module Definition of Done

39. Integration Gateway Module Contract

Owns

Explicitly does not own

Public commands

Public queries

Events published

Events/contracts consumed

Primary permissions

Owned tables

Non-negotiable invariants

Module Definition of Done

40. Notifications Module Contract

Owns

Explicitly does not own

Public commands

Public queries	Part X - Testing and Quality Gates
Events published	52. Test Pyramid by Module
Events/contracts consumed	53. Payroll Release Gate
Primary permissions	54. AI Release Gate
Owned tables	Part XI - DevOps, Releases, Backup and Operations
Non-negotiable invariants	55. CI/CD Pipeline
Module Definition of Done	56. On-Premise Release Package
41. Audit Module Contract	57. Backup and Restore
Owns	58. Observability
Explicitly does not own	Part XII - Developer Work Packages and Integration Governance
Public commands	59. Standard Module Work Package Template
Public queries	60. Inter-Module Change Protocol
Events published	61. Integration Environment Strategy
Events/contracts consumed	62. Code Ownership Matrix
Primary permissions	Part XIII - Product Roadmap Without Becoming Customer-Specific
Owned tables	63. Release Tracks
Non-negotiable invariants	64. AI Rollout Stages
Module Definition of Done	Part XIV - Commercial, IP and Support Guardrails
42. AI Copilot & Learning Center Module Contract	65. Product IP Boundary
Owns	66. Responsibility Matrix Themes
Explicitly does not own	Part XV - Locked Decisions, Deferred Decisions and ADR Backlog
Public commands	67. Locked for v2.0
Public queries	68. Intentionally Deferred
Events published	69. ADRs to Create Before/While Coding
Events/contracts consumed	Part XVI - Developer Checklist Before Merge
Primary permissions	70. Universal Definition of Done
Owned tables	Appendix A - Recommended API Examples
Non-negotiable invariants	Appendix B - Example Module Event Envelopes
Module Definition of Done	Appendix C - Suggested Database Column Conventions
Part VII - Detailed ERD and State Models	Appendix D - High-Risk Domain Invariants
43. Payroll + Compliance ERD	Appendix E - Technology Maintenance Notes (22 August 2026 Baseline)
43.1 Payroll table responsibilities	Appendix F - Full Module Dependency Matrix
43.2 Payroll state machine	F.1 Dependency direction examples
44. Recruitment ERD	Appendix G - Physical Database Catalog v2.0
44.1 Recruitment application state model	Appendix H - State Machine Catalog
44.2 Hire conversion transaction/orchestration	Appendix I - Background Jobs and Schedulers
45. Attendance and Leave Relationships	Appendix J - Frontend Architecture for Independent Module Teams
46. AI Copilot Architecture	Appendix K - ESS / MSS Experience Contract
46.1 AI question classes	Appendix L - Configuration and Entitlement Data Model
46.2 Continuous improvement loop	Appendix M - Error Code and Observability Contract
46.3 AI data model details	Appendix N - Customer Deployment Configuration Checklist
46.4 AI tool contract rules	Appendix O - Recommended Team Assignment Examples
46.5 Gemini/external-provider fallback	Appendix P - Deferred Target-State Modules
Part VIII - Customer Configuration, Entitlements and Extensions	P.1 Performance Management Module
47. Configuration Boundaries	Owns
48. Feature Entitlements	Does not own
49. Vertical Pack Rule	Primary tables
Part IX - Security Architecture	
50. Security Baseline	
51. AI Security and Privacy	

Workforce Platform - Engineering Blueprint v2.0

Product-first, Egypt-first, on-premise-capable workforce platform with payroll/compliance, recruitment, extensible integrations, and a governed AI Copilot.

Canonical engineering document. This document supersedes the previous v1.1 Master Plan and Infrastructure/ERD documents when they conflict. The older documents remain useful historical context, but new engineering work should follow this v2.0 baseline or a newer approved ADR.

Important: this is a product architecture, not a bespoke first-customer project plan. Customer-specific requirements must be implemented through configuration, integration adapters, or isolated extensions whenever possible. The standard product core remains vendor-owned and reusable.

0. How to Use This Document

This blueprint is written specifically so multiple developers can work on different modules in parallel without relying on undocumented assumptions. Before a developer starts a module, they must read:

- **Sections 1-8** for product boundaries and architecture rules.
- **Sections 9-13** for repository, database, API, event, and collaboration contracts.
- The dedicated **Module Contract** section for the module they own.
- **Security, testing, migration, and Definition of Done** sections before opening a production pull request.
- Any ADR referenced by the module. If a required decision is missing, create an ADR instead of inventing an incompatible local pattern.

0.1 Source-of-truth hierarchy

Priority	Artifact	Rule
1	Approved ADR	Overrides this document for the narrow decision it covers.
2	This Engineering Blueprint v2.x	Canonical product and engineering baseline.
3	OpenAPI/JSON schemas and versioned module contracts	Canonical machine-readable integration contract.
4	Database migrations	Canonical physical schema after merge.
5	Code comments / tickets / chat	Informational only; they cannot silently override architecture.

0.2 Required module handoff package

Every module owner must leave the following artifacts so another programmer can continue the work without reverse engineering:

- Module README describing scope, invariants, dependencies, and non-goals.
- Owned table list and ERD.
- OpenAPI endpoints and DTO examples.
- Published/consumed event catalog.
- Permission catalog.
- State machine diagram or table for stateful aggregates.
- Integration tests using real PostgreSQL.
- Seed/test fixtures for other modules.
- Operational notes: scheduled jobs, background processing, retry/idempotency behavior.
- Known limitations and deferred items.

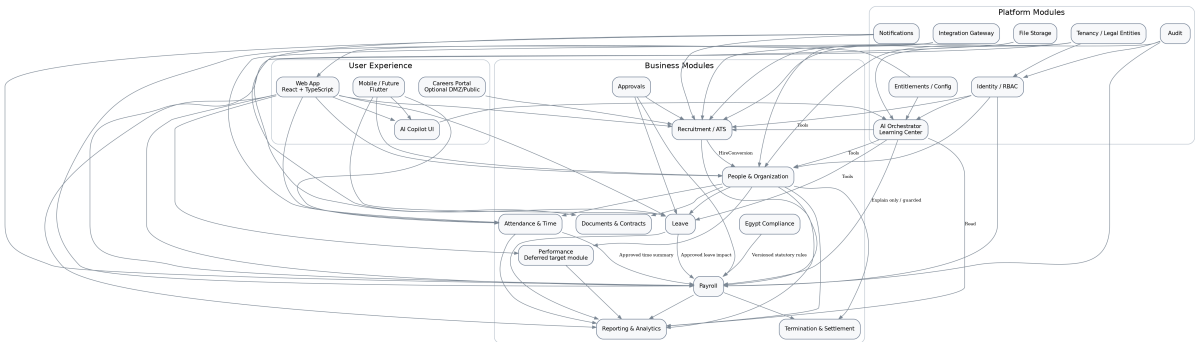
Part I - Product Direction and Non-Negotiable Decisions

1. Product Definition

The product is a **Workforce, Payroll, Compliance and Talent Platform**. It is not merely an HR database and it is not a customer-specific ERP. The platform must support standard workforce capabilities and allow industry/customer extensions without forking the core codebase.

The long-term product is organized into four layers:

Layer	Purpose	Examples
Workforce Core	System of record and daily HR operations	People, organization, contracts, attendance, leave, ESS/MSS
Payroll & Compliance	Egypt-first payroll and legally traceable calculations	Payroll, statutory rules, settlements, explainable payroll, historical reproduction
Talent	Acquire and develop employees	Recruitment/ATS; performance later
Platform & Intelligence	Shared product capabilities	Identity, audit, approvals, integrations, reporting, AI Copilot, learning center, entitlements



Target product module map

2. Product-First Commercial Rule

Customer #1 is the first deployment of the product, not the owner of the product design. The company sells a standard product and then applies configuration, integrations, and bounded extensions.

STANDARD PRODUCT CORE

+

TENANT CONFIGURATION

+

INTEGRATION ADAPTERS

+

OPTIONAL VERTICAL/CUSTOM EXTENSIONS

=

CUSTOMER DEPLOYMENT

A customer request must be classified before code is written:

Class	Meaning	Implementation rule
Product capability	Reusable market need	Add to standard product through normal roadmap.
Configuration	Different value/workflow/policy using existing concepts	Persist as tenant configuration; no fork.
Integration	Connect to customer system	Implement behind connector contract.
Vertical extension	Reusable industry-specific domain	Separate vertical module/package.
One-off customization	Unique behavior with weak reuse	Require commercial approval; isolate behind extension point; never modify core semantics casually.
Bad customization	Customer-specific fork or direct edits to shared core	Reject or redesign.

3. Locked Technology Baseline

Area	Baseline	Reason
Backend	.NET 10 LTS / ASP.NET Core	Long support window, strong typing, enterprise/on-prem suitability, mature tooling.
Database	PostgreSQL 18.x	Open source, robust transactions, JSONB, extensions, strong operational tooling.
ORM	EF Core 10 for transactional domain writes	Migrations, tracking, transactions, maintainability.
Read/query optimization	Dapper or tuned SQL only where measurement justifies it	Avoid ORM contortions for reports without making SQL the default everywhere.
Frontend	React 19.2 + TypeScript + Vite	Internal application; simple SPA deployment, no SSR/SEO requirement.
Reverse proxy	Nginx	TLS termination, static assets, reverse proxy, on-prem friendliness.
Containers	Docker/OCI images; Docker Compose initially	Reproducible on-prem and cloud deployment; Kubernetes deferred.
Observability	OpenTelemetry-compatible logs/metrics/traces	Vendor-neutral supportability.
AI provider	Provider abstraction; Google Gemini is the initial external fallback provider	Avoid hard coupling; allow private/local provider later.
AI retrieval	PostgreSQL + optional pgvector or equivalent vector index	Keep product knowledge close to tenant-controlled data when appropriate.

Technology versions are a **baseline**, not a permanent hard pin. Patch versions must be maintained while supported; major upgrades require an ADR and compatibility test cycle.

4. Architectural Stance

- **Modular Monolith first.** One deployable application and one worker are preferred until measured scaling or independent release requirements justify extraction.
- **Schema-per-module in one PostgreSQL database.** Module ownership is enforced in code review and architecture tests.
- **One codebase, multiple deployment profiles.** On-Prem Dedicated, Private Cloud Dedicated, and Shared SaaS must evolve from the same source.

- **Contract-first module collaboration.** Modules communicate through application contracts/events, not arbitrary table access.
- **Effective dating and immutable financial history.** Historical payroll must remain reproducible.
- **No customer forks.** Product variation uses configuration, entitlements, connectors, and bounded extensions.
- **AI is an orchestrator, not the source of truth.** The LLM may interpret, retrieve, explain, and propose actions; business truth remains in databases, approved policies, and versioned rule packages.
- **Offline-capable core.** On-premise HR/payroll operation must not require the vendor cloud or an external LLM.

5. What Is Expensive Abstraction vs Cheap Guardrail

The team must not confuse future-proofing with building speculative platforms. Preserve low-cost structural decisions that are painful to retrofit, while deferring expensive general-purpose builders until real variation is observed.

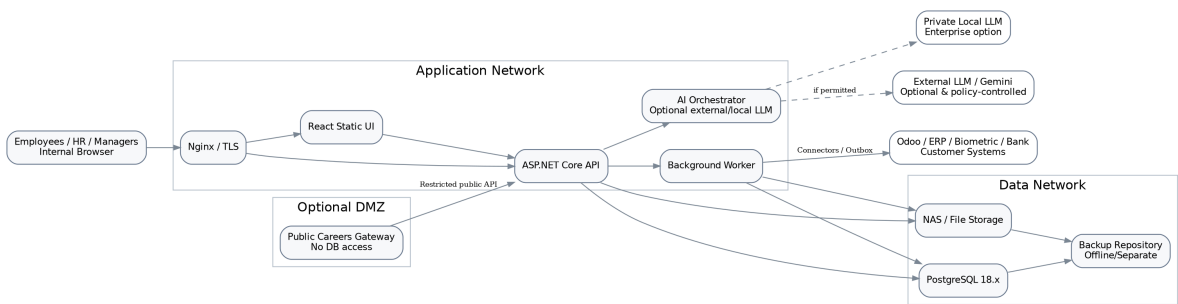
Keep from day one	Defer until demonstrated need
tenant_id / legal_entity_id boundaries	Plugin marketplace
Person/Employee separate from Employment	Generic arbitrary rule language
Effective-dated assignments and compensation	Visual rule designer
Decimal money and financial invariants	Universal no-code configuration studio
Module schemas and explicit contracts	Dozens of microservices
File storage interface (on-prem vs cloud is known)	Kubernetes
Provider interface for external vs private LLM	Autonomous AI agents with broad write access
Rule version metadata and historical snapshots	User-authored statutory payroll formulas

Part II - Deployment and Infrastructure

6. Supported Deployment Profiles

Profile	Database	Files	Internet dependency	Primary use
On-Prem Dedicated	Customer-owned dedicated PostgreSQL	Customer disk/NAS/object store	None for core; optional for external AI/integrations	First enterprise deployments
Private Cloud Dedicated	Dedicated DB per customer or isolated cluster	Cloud object storage	Cloud network	Enterprise hosted option
Shared SaaS	Shared or hybrid tenant storage strategy	Cloud object storage	Cloud	SME scale later

7. On-Premise Reference Topology



On-premise topology

The minimum production shape should preserve network separation even if several roles are colocated on fewer virtual machines for a small installation.

Zone/Role	Components	Inbound	Outbound	Notes
User network	Browsers/mobile clients	N/A	HTTPS to application	No DB access.
DMZ optional	Public careers gateway	HTTPS from internet	Restricted HTTPS to application API	Never direct DB/NAS access.
Application	Nginx, Web assets, ASP.NET API, worker	HTTPS from internal/DMZ	DB, storage, integrations, optional LLM	Application service accounts only.
Data	PostgreSQL, file storage	Only from approved app/backup hosts	Backup/monitoring	Not exposed to user VLAN or internet.
Backup	DB + file backup repository	From backup agents	Optional secure replication	Separate credentials and preferably separate storage/failure domain.

8. Deployment Sizing Profiles

Sizing is workload-driven; employee count alone is not enough. Payroll batch size, attendance event volume, document storage, reporting concurrency, and AI usage matter. Use the following only as starting profiles:

Profile	App	DB	Worker	Storage	Notes
Pilot / small	2-4 vCPU, 8 GB RAM	4 vCPU, 8-16 GB RAM	May share app VM	SSD + NAS	Suitable for validation and small companies; not a contractual performance promise.
Standard enterprise	4-8 vCPU, 16 GB RAM	8 vCPU, 32 GB RAM	Separate process/ container	Fast SSD, separate backup	Recommended baseline when payroll and attendance are business-critical.
High availability later	2+ app nodes	Primary + replica/ managed HA	Redundant workers	Redundant object/ NAS	Only after business RTO/RPO requires it.

9. Public Careers Portal Boundary

If job postings must be public while the main system is internal, expose only a narrow recruitment gateway. The public component may create applications and upload candidate documents through validated APIs, but it cannot query internal HR data or reach PostgreSQL directly.

10. AI Deployment Modes

Mode	LLM location	Data egress	Recommended use
AI Off	None	None	Air-gapped or customers who do not buy AI.
Cloud AI	Gemini/external provider via server-side gateway	Policy-controlled minimal context	Fastest implementation for customers permitting cloud AI.
Private AI	Customer-controlled local/private model	No external egress	Sensitive enterprise/on-prem deployments; premium option.
Hybrid	Cloud for general questions, private tools/data for sensitive operations	Selective	Later advanced profile.

API keys and provider credentials must exist only on the server side. Browser/mobile clients never receive provider secrets.

Part III - Codebase and Multi-Developer Collaboration

11. Recommended Repository Structure

```
repo/  
  src/  
    Workforce.Host.Api/  
    Workforce.Host.Worker/  
    Workforce.SharedKernel/  
    Workforce.BuildingBlocks/  
    Modules/  
      Tenancy/  
        Tenancy.Module/  
        Tenancy.Contracts/  
      Identity/  
        Identity.Module/  
        Identity.Contracts/  
      Organization/  
      People/  
      Documents/  
      Attendance/  
      Leave/  
      Approvals/  
      Payroll/  
      Compliance/  
      Settlement/  
      Recruitment/  
      Reporting/  
      Integrations/  
      Notifications/  
      Audit/  
      Ai/  
  web/  
    src/  
      app/  
      features/  
        people/  
        attendance/  
        leave/  
        payroll/  
        recruitment/  
      ai/  
      shared/  
  tests/  
    Architecture.Tests/  
    EndToEnd.Tests/  
    Modules/<module>.UnitTests/  
    Modules/<module>.IntegrationTests/  
  deploy/  
    docker/  
    nginx/  
    migrations/  
    offline-update/  
  docs/  
    adr/  
    modules/  
    api/
```

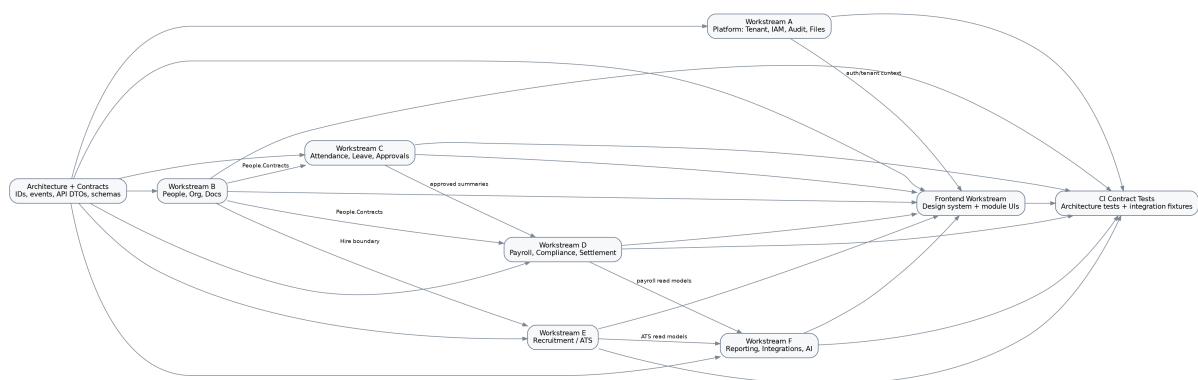
A module may have a separate `Contracts` project when other modules compile against its public commands/query DTOs/events. Small support modules can keep contracts internal until cross-module consumers exist.

12. Dependency Rules

The architecture test suite must enforce the following:

- Domain code does not reference EF Core, HTTP, external providers, or another module implementation.
- A module may reference another module only through its Contracts assembly or approved shared kernel primitives.
- No module directly invokes another module's DbContext.
- Reporting may use approved read models/views and optimized SQL; it cannot mutate business module tables.
- AI may call backend tools/query contracts; it cannot use a write-capable database connection.
- Integrations translate external models through adapters/anti-corruption layers; external DTOs do not leak into domain entities.
- SharedKernel contains only truly universal primitives (IDs, Money, DateRange, Result, tenant context), not business logic from a specific module.

13. Parallel Workstream Model



Parallel workstreams

Multiple developers can start after foundational contracts are frozen enough to mock. Dependent modules should consume interface contracts and test fixtures, not wait for every upstream screen to be finished.

Workstream	Primary modules	Can begin when	Key contracts
A - Platform	Tenancy, Identity, Audit, Files, Entitlements	Repository baseline exists	TenantContext, CurrentUser, AuditSink, FileStorage
B - Workforce Core	Organization, People, Documents/Contracts	Tenant/legal entity contracts frozen	EmployeeId, EmploymentId, Assignment snapshot
C - Time	Attendance, Leave, initial approval patterns	Employment contract frozen	Employment lookup, approved time/leave outputs
D - Payroll	Payroll, Compliance, Settlement	Employment + time summary contracts frozen	PayrollInput, RuleVersion, FinalizedPayroll
E - Talent	Recruitment/ATS	Tenant/org/position contracts frozen	Requisition, Candidate/ Application, HireConversion
F - Intelligence/Integration	Reporting, connectors, AI	Public query/tool contracts exist	Read models, events, tool schemas
Frontend	Design system and module UIs	OpenAPI mocks available	Generated API client + permission metadata

14. Git and Pull Request Rules

- Use trunk-based development with short-lived branches; avoid months-long module branches.
- Every module has CODEOWNERS. Cross-module contract or schema changes require the owner of both producer and consumer modules.
- PRs that change public contracts must include migration/compatibility notes and update contract tests.
- PRs must remain module-focused. Refactors across six modules require an explicit architecture ticket/ADR.
- No direct production database edits. Every schema change is an idempotent migration committed with code.
- CI must run architecture tests, affected module unit/integration tests, OpenAPI validation, migrations from clean DB, and migration upgrade from previous supported release.

15. Contract-First Workflow for Developers

1. Define/confirm module boundary.
2. Define IDs, commands, queries, events and permission names.
3. Define owned tables and state transitions.
4. Publish mock/stub contracts and sample payloads.
5. Consumers implement against contracts/mocks.
6. Module owner implements domain + persistence.
7. Add integration/contract tests.
8. Replace mocks in integrated environment.
9. Only then add optimizations or additional abstractions.

Part IV - Database Architecture and Relationship Rules

16. PostgreSQL Schema Ownership

Schema	Owner module	Typical tables
platform	Tenancy/Entitlements	tenants, legal_entities, tenant_settings, feature_entitlements
iam	Identity	users, identities, roles, permissions, role_assignments, sessions
org	Organization	branches, departments, positions, cost_centers, work_locations
people	People	employees, employments, employee_assignments, contacts, bank_accounts, employee_documents links
documents	Documents	files, document_types, templates, signatures metadata
time	Attendance	attendance_events, attendance_days, shifts, schedules, period_summaries
leave	Leave	leave_types, policies, balances, requests, transactions, payroll_impacts
approvals	Approvals	approval_policies, instances, steps, actions/delegations
payroll	Payroll	components, assignments, periods, runs, inputs, line_items, payslips, payment_batches
compliance	Compliance	rule_sets, rule_versions, source_documents, executions, golden_cases
settlement	Settlement	termination_cases, settlement_runs, settlement_lines
recruitment	Recruitment	requisitions, postings, candidates, applications, stages, interviews, offers, hire_conversions
reporting	Reporting	read models/materialized views metadata, export jobs
integration	Integrations	connectors, configs, outbox, inbox, sync_runs, external_mappings
notifications	Notifications	templates, messages, deliveries, preferences
audit	Audit	audit_events, security_events, access_log metadata
ai	AI	conversations, interactions, feedback, knowledge, learning_items, evals, usage, action_proposals

17. Database Rules

- Use `uuid` primary keys generated as UUIDv7 by the application where practical. Public human-facing codes are separate business keys.
- All tenant-owned rows carry `tenant_id`; legal-entity-specific rows also carry `legal_entity_id` where the distinction matters.
- Use `numeric(19,4)` for monetary amounts unless a domain needs a stricter scale. Never use floating point for money.
- Use `date` for business calendar dates and `timestampz` for instants. Store UTC instants; render in configured timezone.
- Effective-dated ranges use `effective_from` inclusive and `effective_to` exclusive/null for open-ended. Prevent unintended overlap with constraints or transactional validation.
- Financial/finalized payroll history is immutable. Corrections create new runs/adjustments rather than rewriting history.
- Hard delete is prohibited for payroll, audit, approved leave, employment history, compliance versions, and finalized recruitment decisions that must be retained. Candidate data is subject to configured privacy/retention deletion or anonymization.
- JSONB is allowed for sparse provider payloads, immutable snapshots, rule execution traces, and extension metadata; it is not a substitute for modeling core relational fields.
- Indexes must follow real query patterns. Every FK used for joins and every major tenant/date filter requires index review.
- Uniqueness must generally include tenant/legal-entity scope, e.g. `(tenant_id, employee_no)`.

18. Cross-Module Foreign Keys and Ownership

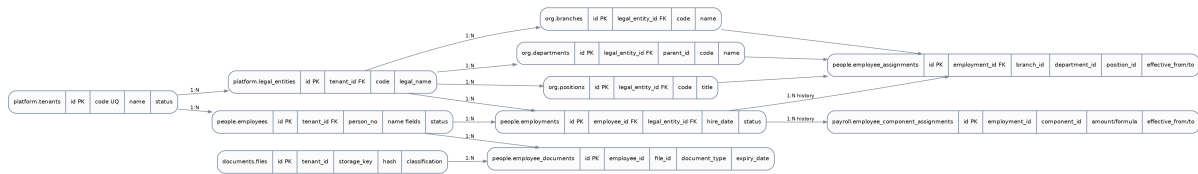
A physical foreign key does not grant ownership. The rule is **one writer per table**. A module never updates another module's table even if the database allows a relationship.

Relationship type	Database FK?	Runtime access rule
Within same module/schema	Yes by default	Normal aggregate access through module DbContext.
To stable platform roots (tenant/legal entity)	Yes	Read identifier/context; writes remain platform-owned.
To stable workforce roots (employee/employment)	Allowed when integrity is high-value	Dependent module stores ID and calls People contract for current business data.
Between volatile business modules	Prefer logical ID/event snapshot unless ADR approves FK	No direct joins in transactional code.
Reporting joins	Views/read-model SQL may join approved tables	Read-only reporting connection; never write.
AI	No direct business-table write FK behavior	AI uses tools/read models; not a database client.

19. Snapshot vs Reference Rule

Use a reference when current truth is required; use a snapshot when historical reproducibility is required. Payroll is the clearest example: a run references an `employment_id` but also snapshots the salary inputs, attendance totals, leave impact, rule versions, and calculation outputs used at that time.

20. Core Workforce ERD



Core workforce ERD

Critical cardinalities:

- Tenant 1:N LegalEntity.
- Tenant 1:N Employee identity records.
- Employee 1:N Employment. Rehire or concurrent employment is possible without duplicating the human record.
- Employment 1:N Assignment history.
- LegalEntity/Branch/Department/Position 1:N Assignment references.
- Employment 1:N effective-dated compensation assignments.
- Employee 1:N employee-document links; file binary metadata is owned by Documents.

21. Migration Ownership

- Each module owns migrations for its schema and may use its own EF Core migration history table.
- Platform/Tenancy migrations execute before business modules.
- Cross-module schema changes require an explicit dependency order in the release manifest.
- A migration must support upgrade from the previous supported release and be tested against representative production-scale data.
- Destructive changes use expand/migrate/contract across releases when data or compatibility risk is significant.

Part V - API, Events, Permissions and Shared Contracts

22. API Conventions

- REST first with OpenAPI. Base route: `/api/v1/{module}/...`
- Use request/response DTOs; never expose EF entities directly.
- Commands return identifiers/state/version metadata; queries may return purpose-built projections.
- Use RFC 7807-style problem details for errors with stable product error codes.
- Every request executes in explicit tenant/user context. Tenant identity must not be trusted solely from a client-supplied body field.
- Use optimistic concurrency for user-edited records (`version`/ETag or equivalent).
- Pagination, sorting, filtering, and export limits must be standardized across modules.
- Idempotency keys are required for externally retried create/action endpoints where duplicates are harmful.

23. Domain Events vs Integration Events

Type	Scope	Durability	Example
Domain event	Inside module transaction/ application flow	May be in-memory	EmploymentStatusChanged domain reaction.
Module/application event	Between modules in modular monolith	Outbox when side effect must survive failure	PayrollFinalized consumed by Reporting/Integration.
External integration event/ webhook	Outside product boundary	Durable, versioned, retryable	employee.updated.v1 to external ERP.

Do not publish entire database rows. Events contain stable identifiers and the minimum immutable facts needed by consumers.

24. Permission Naming

```
<module>.<resource>.<action>
```

Examples:

```
people.employee.read
people.employee.update
payroll.run.calculate
payroll.run.finalize
recruitment.offer.approve
ai.copilot.use
ai.learning.review
ai.action.execute_sensitive
```

Permissions may also carry data scope: self, direct reports, department, legal entity, tenant. Scope evaluation happens in the backend before data is retrieved for both human UI and AI tools.

25. Shared Identifiers and Value Objects

Primitive	Rule
TenantId	Required in all tenant-bound operations.
LegalEntityId	Required when legal employer matters.
EmployeeId	Human/person master identity within tenant.
EmploymentId	Specific employment relationship; payroll/time/leave primarily bind here.
Money	Amount + currency; decimal only.
DateRange	Inclusive start / exclusive end semantics.
EffectivePeriod	Business effective range with overlap rules.
ActorContext	User/service identity + tenant + legal entity + permission scopes.
CorrelationId	Flows through API, jobs, events, integrations, and AI tool calls.

Part VI - Module Contracts

The following module packs are the minimum boundary contract. Detailed endpoint schemas live in OpenAPI and code, but those machine contracts must not contradict these ownership rules.

26. Tenancy & Legal Entities Module Contract

Schema: platform

Purpose: Own the customer boundary, legal employers, tenant settings, deployment identity and feature entitlements.

Owns

- Tenant
- LegalEntity
- TenantSetting
- FeatureEntitlement
- DeploymentIdentity

Explicitly does not own

- Employee/person data
- Users/passwords
- Payroll settings beyond feature entitlement

Public commands

- CreateTenant (vendor/admin only)
- CreateLegalEntity
- UpdateLegalEntity
- SetTenantSetting
- GrantFeatureEntitlement

Public queries

- GetTenantContext
- ListLegalEntities
- GetEntitlements

Events published

- TenantCreated
- LegalEntityCreated

- LegalEntityUpdated
- FeatureEntitlementChanged

Events/contracts consumed

- None; foundation module

Primary permissions

- platform.tenant.read
- platform.legal_entity.manage
- platform.entitlement.manage

Owned tables

- platform.tenants
- platform.legal_entities
- platform.tenant_settings
- platform.feature_entitlements
- platform.deployment_instances

Non-negotiable invariants

- Tenant code is globally unique in an installation.
- Legal entity code is unique within tenant.
- Business modules cannot create tenants implicitly.
- Feature entitlement does not bypass authorization.

Module Definition of Done

- Integration tests for tenant scoping
- Seed/bootstrap path for first tenant
- No business module can run without TenantContext

27. Identity & Access Module Contract

Schema: iam

Purpose: Authentication, user accounts, roles, permissions, MFA/SSO hooks and scoped authorization.

Owns

- User
- ExternalIdentity
- Role
- Permission
- RoleAssignment

- Session
- MfaCredential

Explicitly does not own

- Employee HR record
- Business approval role semantics
- Customer directory source of truth when federated

Public commands

- CreateUser
- DisableUser
- AssignRole
- RemoveRole
- ConfigureFederation
- ResetMfa

Public queries

- GetCurrentUser
- GetEffectivePermissions
- ListUsers
- ResolveUserScopes

Events published

- UserCreated
- UserDisabled
- RoleAssignmentChanged
- SecurityCredentialChanged

Events/contracts consumed

- EmployeeCreated optionally to offer account-link workflow; LegalEntityCreated for scopes

Primary permissions

- iam.user.manage
- iam.role.manage
- iam.security.manage

Owned tables

- iam.users
- iam.user_identities
- iam.roles

- `iam.permissions`
- `iam.role_permissions`
- `iam.user_role_assignments`
- `iam.sessions`
- `iam.mfa_credentials`

Non-negotiable invariants

- Employee and User are separate concepts.
- Authorization is backend-enforced.
- AI inherits the calling user scope; no AI superuser shortcut.
- Break-glass actions are audited.

Module Definition of Done

- Permission matrix tests
- Tenant isolation tests
- Account lock/disable tests
- SSO adapter interface documented

28. Organization Module Contract

Schema: `org`

Purpose: Model branches, departments, positions, cost centers, work locations and organizational hierarchy.

Owns

- Branch
- Department
- Position
- CostCenter
- WorkLocation

Explicitly does not own

- Employee assignment history
- Payroll components
- Recruitment applications

Public commands

- `CreateDepartment`
- `MoveDepartment`
- `CreatePosition`
- `UpdatePosition`

- CreateBranch
- CreateCostCenter

Public queries

- GetOrgTree
- ListPositions
- ResolveDepartmentHierarchy
- GetWorkLocation

Events published

- DepartmentCreated
- DepartmentChanged
- PositionCreated
- PositionChanged
- BranchChanged

Events/contracts consumed

- LegalEntityCreated

Primary permissions

- org.structure.read
- org.structure.manage

Owned tables

- org.branches
- org.departments
- org.positions
- org.cost_centers
- org.work_locations

Non-negotiable invariants

- Codes unique within intended legal-entity scope.
- Hierarchy cannot contain cycles.
- Historical employee assignments are not rewritten when org names change.

Module Definition of Done

- Hierarchy cycle tests
- Effective reporting labels strategy documented
- Org seed/import template

29. People Module Contract

Schema: `people`

Purpose: Canonical employee/person master and employment relationships, including effective-dated assignments and contact/bank details.

Owns

- Employee
- Employment
- EmployeeAssignment
- EmployeeContact
- EmployeeBankAccount
- EmergencyContact

Explicitly does not own

- Authentication credentials
- Attendance events
- Leave balances
- Payroll run results
- Recruitment candidate before hire

Public commands

- `CreateEmployee`
- `CreateEmployment`
- `ChangeAssignment`
- `ChangeEmploymentStatus`
- `UpdatePersonalDetails`
- `UpdateBankAccount`
- `RehireEmployee`

Public queries

- `GetEmployee`
- `GetEmployment`
- `GetEmploymentSnapshot(asOf)`
- `SearchEmployees`
- `GetManagerChain`

Events published

- `EmployeeCreated`
- `EmployeeUpdated`

- `EmploymentCreated`
- `EmploymentStatusChanged`
- `AssignmentChanged`
- `BankAccountChanged`

Events/contracts consumed

- `HireConversionRequested` from Recruitment (through application command)
- Organization reference changes

Primary permissions

- `people.employee.read`
- `people.employee.create`
- `people.employee.update`
- `people.employment.manage`
- `people.bank_account.manage`

Owned tables

- `people.employees`
- `people.employments`
- `people.employee_assignments`
- `people.employee_contacts`
- `people.employee_bank_accounts`
- `people.emergency_contacts`
- `people.employee_tags`

Non-negotiable invariants

- Employee is not Employment.
- Employee number unique per tenant/legal-entity policy.
- Assignment history is effective-dated and non-overlapping for the primary assignment.
- Employment termination does not delete the Employee.

Module Definition of Done

- As-of-date snapshot tests
- Rehire test
- Concurrent employment policy test
- No direct salary column on employees

30. Documents & Contracts Module Contract

Schema: `documents + people links`

Purpose: File metadata/storage abstraction, document types/templates, employee documents, contracts and expiry tracking.

Owns

- FileMetadata
- DocumentType
- DocumentTemplate
- ContractDocument metadata
- SignatureMetadata

Explicitly does not own

- Binary storage implementation details outside IFileStorage
- Employment status
- Candidate resume ownership before hire (Recruitment links it)

Public commands

- UploadFile
- AttachEmployeeDocument
- CreateContractDocument
- ReplaceDocumentVersion
- RecordSignature
- ArchiveDocument

Public queries

- GetFileMetadata
- ListEmployeeDocuments
- ListExpiringDocuments
- GetContractVersion

Events published

- FileStored
- EmployeeDocumentAttached
- DocumentExpiring
- ContractSigned

Events/contracts consumed

- EmployeeCreated
- EmploymentCreated

Primary permissions

- `documents.file.read`
- `documents.employee_document.manage`
- `documents.contract.manage`

Owned tables

- `documents.files`
- `documents.document_types`
- `documents.document_templates`
- `documents.document_versions`
- `people.employee_documents`
- `people.employment_contracts`

Non-negotiable invariants

- Binary content is addressed through storage key, not stored casually in business rows.
- Hash/size/mime metadata retained.
- Sensitive document access is separately authorized and audited.
- Versioned contracts are not overwritten.

Module Definition of Done

- Local/NAS storage adapter
- Virus-scan hook interface
- Expiry query/index
- Permission tests for sensitive documents

31. Attendance & Time Module Contract

Schema: `time`

Purpose: Capture raw attendance events, schedules/shifts, daily interpretation, corrections, approvals and payroll-ready period summaries.

Owns

- `AttendanceEvent`
- `Shift`
- `WorkSchedule`
- `AttendanceDay`
- `AttendanceCorrection`
- `AttendancePeriodSummary`

Explicitly does not own

- Employment master
- Leave request truth
- Payroll calculation

Public commands

- ImportAttendanceEvents
- RecordManualEvent
- AssignSchedule
- RecalculateAttendanceDay
- RequestCorrection
- ApprovePeriodSummary

Public queries

- GetAttendanceDay
- GetAttendanceCalendar
- GetPeriodSummary
- GetLateAbsentEmployees

Events published

- AttendanceEventRecorded
- AttendanceDayCalculated
- AttendanceCorrectionApproved
- AttendancePeriodApproved

Events/contracts consumed

- EmploymentCreated/StatusChanged
- AssignmentChanged
- ApprovedLeaveChanged (for interpretation)

Primary permissions

- time.attendance.read
- time.attendance.edit
- time.correction.approve
- time.period.approve

Owned tables

- time.attendance_events
- time.shifts

- `time.work_schedules`
- `time.schedule_assignments`
- `time.attendance_days`
- `time.attendance_corrections`
- `time.attendance_period_summaries`

Non-negotiable invariants

- Raw imported events are retained even if interpretation changes.
- Manual corrections require actor/reason.
- Payroll consumes approved period summary/snapshot, not arbitrary live punches.
- Time zone/work location rules are explicit.

Module Definition of Done

- Biometric import idempotency tests
- Overnight shift tests
- DST/timezone tests where relevant
- Period approval lock tests

32. Leave Module Contract

Schema: `Leave`

Purpose: Leave types/policies, balances, accruals, requests, approvals, transactions and payroll impact.

Owns

- `LeaveType`
- `LeavePolicy`
- `LeaveBalance`
- `LeaveRequest`
- `LeaveTransaction`
- `LeavePayrollImpact`

Explicitly does not own

- General approval infrastructure implementation
- Payroll deduction calculation
- Employment master

Public commands

- `CreateLeaveRequest`
- `CancelLeaveRequest`
- `ApproveLeave`

- `RejectLeave`
- `PostLeaveAdjustment`
- `AccrueLeave`
- `CloseLeaveYear`

Public queries

- `GetLeaveBalance`
- `ListLeaveRequests`
- `CalculateRequestedUnits`
- `GetTeamCalendar`

Events published

- `LeaveRequested`
- `LeaveApproved`
- `LeaveRejected`
- `LeaveCancelled`
- `LeaveBalanceChanged`
- `LeavePayrollImpactCreated`

Events/contracts consumed

- `EmploymentCreated/Terminated`
- Attendance period events optionally
- `ApprovalActionCompleted`

Primary permissions

- `leave.request.self`
- `leave.balance.read`
- `leave.request.approve`
- `leave.adjustment.manage`

Owned tables

- `leave.leave_types`
- `leave.leave_policies`
- `leave.policy_assignments`
- `leave.leave_balances`
- `leave.leave_requests`
- `leave.leave_transactions`
- `leave.leave_payroll_impacts`

Non-negotiable invariants

- Balances derive from auditable transactions, not only one mutable number.
- Approved leave cannot be silently edited; use amendment/cancel flow.
- Statutory minimums are validated against Compliance policy where applicable.

Module Definition of Done

- Accrual golden cases
- Cross-year tests
- Half-day/unit tests
- Approval/cancellation concurrency tests

33. Approvals Module Contract

Schema: `approvals`

Purpose: Provide a small shared approval execution model without prematurely building a universal workflow product.

Owns

- `ApprovalPolicy`
- `ApprovalInstance`
- `ApprovalStep`
- `ApprovalAction`
- `Delegation`

Explicitly does not own

- Business object state
- Business validation
- Visual generic workflow designer in v1

Public commands

- `StartApproval`
- `ApproveStep`
- `RejectStep`
- `CancelApproval`
- `DelegateApproval`

Public queries

- `GetApprovalStatus`
- `GetPendingApprovalsForUser`

Events published

- `ApprovalStarted`
- `ApprovalStepApproved`
- `ApprovalRejected`
- `ApprovalCompleted`
- `ApprovalCancelled`

Events/contracts consumed

- Module-specific request to start approval

Primary permissions

- `approvals.inbox.read`
- `approvals.action.execute`
- `approvals.policy.manage`

Owned tables

- `approvals.approval_policies`
- `approvals.approval_instances`
- `approvals.approval_steps`
- `approvals.approval_actions`
- `approvals.delegations`

Non-negotiable invariants

- Approvals do not directly mutate Leave/Payroll/Recruitment tables.
- Calling module decides business state after approved/rejected result.
- Policy is explicit and versioned enough to explain who approved what.

Module Definition of Done

- Simple sequential approval first
- Delegation audit
- Idempotent repeated action handling
- No visual builder required for initial implementation

34. Payroll Module Contract

Schema: `payroll`

Purpose: Calculate, review, finalize, explain, reproduce and export payroll using immutable snapshots and versioned compliance rules.

Owns

- PayrollComponent
- EmployeeComponentAssignment
- PayrollPeriod
- PayrollRun
- PayrollRunEmployee
- PayrollInput
- PayrollLineItem
- Payslip
- PaymentBatch

Explicitly does not own

- Statutory rule authoring truth (Compliance owns versions)
- Attendance raw punches
- Leave balance
- Employee master

Public commands

- CreatePayrollPeriod
- CreatePayrollRun
- LoadPayrollInputs
- CalculatePayroll
- RecalculateEmployee
- ReviewPayroll
- FinalizePayroll
- CreateAdjustmentRun
- PublishPayslips
- CreatePaymentBatch

Public queries

- GetPayrollRun
- GetEmployeeCalculationTrace
- ComparePayrollPeriods
- GetPayslip
- GetPayrollVariance

Events published

- PayrollRunCreated
- PayrollCalculated

- PayrollEmployeeCalculated
- PayrollFinalized
- PayslipPublished
- PaymentBatchCreated

Events/contracts consumed

- Employment/Assignment/Compensation snapshots
- AttendancePeriodApproved
- LeavePayrollImpactCreated
- ComplianceRulePublished

Primary permissions

- payroll.run.read
- payroll.run.calculate
- payroll.run.review
- payroll.run.finalize
- payroll.payslip.publish
- payroll.payment.export

Owned tables

- payroll.payroll_components
- payroll.employee_component_assignments
- payroll.payroll_periods
- payroll.payroll_runs
- payroll.payroll_run_employees
- payroll.payroll_inputs
- payroll.payroll_line_items
- payroll.payslips
- payroll.payment_batches
- payroll.adjustment_links

Non-negotiable invariants

- No floating-point money.
- Finalized run is immutable.
- Every line is traceable to input/component/rule or explicit manual adjustment.
- Recalculation is idempotent for same immutable inputs and calculation version.
- Historical run does not depend on current employee salary or current law.

Module Definition of Done

- Golden payroll suite
- Parallel-run comparison import
- Finalization lock tests
- Explainability endpoint
- Variance report
- Bank/export abstraction

35. Egypt Compliance Module Contract

Schema: `compliance`

Purpose: Version, test, publish and execute statutory policies for Egyptian labor/payroll obligations without hiding legal logic in scattered code.

Owns

- RuleSet
- RuleVersion
- ComplianceSource
- RuleExecution metadata
- GoldenCase
- RulePackage

Explicitly does not own

- Customer discretionary allowances
- Payroll run state
- Legal advice outside encoded/approved scope

Public commands

- `DraftRuleVersion`
- `ValidateRuleVersion`
- `PublishRuleVersion`
- `RetireRuleVersion`
- `ImportRulePackage`

Public queries

- `ResolveEffectiveRule`
- `GetRuleVersion`
- `GetLegalSourceReference`
- `GetRuleChangeHistory`

Events published

- `ComplianceRuleDrafted`
- `ComplianceRulePublished`
- `ComplianceRuleRetired`
- `CompliancePackageInstalled`

Events/contracts consumed

- Vendor/offline signed update package

Primary permissions

- `compliance.rule.read`
- `compliance.rule.review`
- `compliance.rule.publish`

Owned tables

- `compliance.rule_sets`
- `compliance.rule_versions`
- `compliance.rule_sources`
- `compliance.golden_cases`
- `compliance.rule_packages`
- `compliance.rule_executions`

Non-negotiable invariants

- Published version is immutable.
- Effective dates cannot create ambiguous active versions for same scope without explicit priority.
- Statutory rule changes require golden tests and reviewer sign-off.
- Customer cannot casually edit vendor-controlled statutory logic.

Module Definition of Done

- Signed package import
- Version resolution tests
- Source-reference metadata
- Golden legal/payroll cases
- Rollback/disable strategy for bad rule release

36. Termination & Settlement Module Contract

Schema: `settlement`

Purpose: Model termination reasons and calculate final settlement using employment, payroll, leave and compliance inputs.

Owns

- TerminationCase
- SettlementRun
- SettlementInput
- SettlementLine
- SettlementApproval

Explicitly does not own

- Termination master data history outside case
- Payroll historical runs
- Legal rule versions

Public commands

- OpenTerminationCase
- CalculateSettlement
- ReviewSettlement
- FinalizeSettlement
- CancelTerminationCase

Public queries

- GetTerminationCase
- ExplainSettlement
- GetOutstandingSettlementItems

Events published

- TerminationCaseOpened
- SettlementCalculated
- SettlementFinalized
- EmploymentTerminationRequested

Events/contracts consumed

- Employment snapshot
- Leave balance/transactions
- Payroll history
- Compliance rules

Primary permissions

- settlement.case.manage
- settlement.calculate

- `settlement.finalize`

Owned tables

- `settlement.termination_cases`
- `settlement.settlement_runs`
- `settlement.settlement_inputs`
- `settlement.settlement_lines`

Non-negotiable invariants

- Final settlement is not a single universal formula.
- Inputs/reason/contract context are snapshotted.
- Finalized settlement is immutable and auditable.

Module Definition of Done

- Multiple termination reason tests
- Unused leave/payroll integration tests
- Explanation trace
- Employment termination orchestration

37. Recruitment / ATS Module Contract

Schema: `recruitment`

Purpose: Manage requisitions, postings, candidates, applications, pipeline history, interviews, offers and controlled conversion into People.

Owns

- JobRequisition
- JobPosting
- Candidate
- Application
- Pipeline
- Interview
- Evaluation
- Offer
- HireConversion

Explicitly does not own

- Employee after hire
- Payroll salary assignment after employment creation
- Authentication account

Public commands

- `CreateRequisition`
- `ApproveRequisition`
- `PublishJob`
- `CreateCandidate`
- `ApplyCandidate`
- `MoveApplicationStage`
- `ScheduleInterview`
- `SubmitEvaluation`
- `CreateOffer`
- `AcceptOffer`
- `ConvertCandidateToHire`

Public queries

- `SearchCandidates`
- `GetPipeline`
- `GetApplication`
- `GetInterviewSchedule`
- `GetOffer`
- `RecruitmentFunnel`

Events published

- `RequisitionApproved`
- `ApplicationCreated`
- `ApplicationStageChanged`
- `InterviewCompleted`
- `OfferAccepted`
- `CandidateHireConverted`

Events/contracts consumed

- Position/Org updates
- Approval results
- Employee/Employment creation result during conversion

Primary permissions

- `recruitment.requisition.manage`
- `recruitment.candidate.read`
- `recruitment.application.move`
- `recruitment.interview.evaluate`

- `recruitment.offer.manage`
- `recruitment.hire.convert`

Owned tables

- `recruitment.job_requisitions`
- `recruitment.job_postings`
- `recruitment.recruitment_pipelines`
- `recruitment.pipeline_versions`
- `recruitment.recruitment_stages`
- `recruitment.candidates`
- `recruitment.candidate_documents`
- `recruitment.applications`
- `recruitment.application_stage_events`
- `recruitment.interviews`
- `recruitment.interview_participants`
- `recruitment.interview_evaluations`
- `recruitment.offers`
- `recruitment.offer_versions`
- `recruitment.offer_components`
- `recruitment.hire_conversions`

Non-negotiable invariants

- Candidate is not Employee.
- Application stage history is append-only/auditable.
- Accepted offer version is preserved.
- Hire conversion is idempotent and creates People records only through People application contract.
- Candidate retention/deletion respects privacy policy.

Module Definition of Done

- Pipeline state tests
- Duplicate candidate strategy
- Offer versioning
- Hire conversion transaction/orchestration tests
- Public gateway threat tests

38. Reporting & Analytics Module Contract

Schema: `reporting`

Purpose: Provide read-optimized, permission-aware operational and management reporting without letting report queries leak into transactional modules.

Owns

- ReportDefinition
- ExportJob
- ScheduledReport
- ReadModel metadata/materialization jobs

Explicitly does not own

- Business source-of-truth rows
- Write operations

Public commands

- CreateExportJob
- ScheduleReport
- RefreshReadModel

Public queries

- EmployeeSummary
- AttendanceAnalytics
- PayrollVariance
- RecruitmentFunnel
- ComplianceAuditReport

Events published

- ReportGenerated
- ExportCompleted

Events/contracts consumed

- Business module events or reads approved source views

Primary permissions

- reporting.report.read
- reporting.export.create
- reporting.sensitive_payroll.read

Owned tables

- reporting.report_definitions
- reporting.export_jobs
- reporting.scheduled_reports
- reporting.read_model_versions

Non-negotiable invariants

- Reporting connection is read-only to business schemas.
- Every query applies tenant + permission scope.
- Sensitive payroll reports have separate permissions.

Module Definition of Done

- Representative query performance tests
- Export row limits
- Permission tests
- Materialized view refresh strategy if used

39. Integration Gateway Module Contract

Schema: `integration`

Purpose: Connect Odoo/ERP, biometric devices, banks and external systems while isolating external schemas from product domain models.

Owns

- Connector
- ConnectorConfig
- ExternalMapping
- SyncRun
- OutboxMessage
- InboxMessage
- WebhookSubscription

Explicitly does not own

- Business source data
- External system master truth except stored mapping/state

Public commands

- `ConfigureConnector`
- `RunSync`
- `RetrySync`
- `MapExternalId`
- `PublishWebhook`

Public queries

- `GetConnectorHealth`
- `GetSyncHistory`

- `GetFailedMessages`

Events published

- `ConnectorConfigured`
- `SyncCompleted`
- `SyncFailed`
- `ExternalMessageReceived`

Events/contracts consumed

- `PayrollFinalized`
- `EmployeeChanged`
- Attendance import payloads
- Recruitment events as configured

Primary permissions

- `integration.connector.manage`
- `integration.sync.run`
- `integration.error.review`

Owned tables

- `integration.connectors`
- `integration.connector_configs`
- `integration.external_mappings`
- `integration.sync_runs`
- `integration.outbox_messages`
- `integration.inbox_messages`
- `integration.webhook_subscriptions`

Non-negotiable invariants

- External retries are idempotent.
- Secrets encrypted/not returned in plaintext APIs.
- External DTOs are translated at boundary.
- A connector failure must not roll back finalized payroll.

Module Definition of Done

- Retry/backoff tests
- Idempotency tests
- Dead-letter/error review UI contract
- At least one reference connector

40. Notifications Module Contract

Schema: `notifications`

Purpose: Deliver email/SMS/WhatsApp/push/in-app notifications through provider adapters and tenant templates.

Owns

- `NotificationTemplate`
- `NotificationMessage`
- `DeliveryAttempt`
- `NotificationPreference`

Explicitly does not own

- Business event source
- Provider account truth

Public commands

- `QueueNotification`
- `RetryDelivery`
- `ManageTemplate`
- `SetPreference`

Public queries

- `GetDeliveryStatus`
- `GetUserNotifications`

Events published

- `NotificationQueued`
- `NotificationDelivered`
- `NotificationFailed`

Events/contracts consumed

- Business events selected by policy

Primary permissions

- `notifications.template.manage`
- `notifications.delivery.read`

Owned tables

- `notifications.templates`
- `notifications.messages`

- `notifications.delivery_attempts`
- `notifications.preferences`

Non-negotiable invariants

- Notification failure does not corrupt source transaction.
- Sensitive values are minimized in external channels.
- Templates are localized.

Module Definition of Done

- SMTP adapter first
- Retry/idempotency
- Template localization
- Sensitive-data redaction tests

41. Audit Module Contract

Schema: `audit`

Purpose: Append high-value business/security audit events with actor, tenant, correlation and before/after summaries.

Owns

- `AuditEvent`
- `SecurityEvent`

Explicitly does not own

- Application debug logs
- Business domain state

Public commands

- `AppendAuditEvent` (internal contract only)

Public queries

- `SearchAuditEvents`
- `GetEntityHistory`

Events published

- `No business dependency; audit itself is terminal`

Events/contracts consumed

- Security/business actions from all modules

Primary permissions

- `audit.read`
- `audit.security.read`

Owned tables

- `audit.audit_events`
- `audit.security_events`

Non-negotiable invariants

- Application users cannot delete audit history.
- Secrets/passwords/raw sensitive document bytes never enter audit payload.
- Finalized payroll and sensitive reads/actions generate appropriate audit entries.

Module Definition of Done

- Append-only permissions
- Retention/archive plan
- Correlation search
- Sensitive payload filter

42. AI Copilot & Learning Center Module Contract

Schema: `ai`

Purpose: Provide Ask, Analyze, Explain and guarded Act experiences while learning from reviewed user interactions without turning unverified model output into business truth.

Owns

- `AiConversation`
- `AiInteraction`
- `AiFeedback`
- `KnowledgeSource`
- `KnowledgeDocument/Chunk`
- `LearningItem`
- `EvalCase`
- `EvalRun`
- `PromptVersion`
- `SemanticCache`
- `ModelUsage`
- `ActionProposal`
- `ActionExecution metadata`
- `TenantAiSettings`

Explicitly does not own

- Employee/payroll truth
- Statutory rules
- Business-table writes
- Authentication permissions

Public commands

- AskCopilot
- SubmitFeedback
- ReviewLearningItem
- ApproveKnowledge
- RunEvalSuite
- ProposeAction
- ConfirmAction
- ConfigureAiTenantPolicy

Public queries

- GetConversation
- GetLearningQueue
- GetAiUsage
- GetEvalResults
- SearchApprovedKnowledge

Events published

- AiInteractionCompleted
- AiFeedbackSubmitted
- KnowledgeApproved
- AiActionProposed
- AiActionExecuted
- AiEvalFailed

Events/contracts consumed

- Approved backend tool schemas
- Reporting read models
- Knowledge updates
- Tenant/permission context

Primary permissions

- ai.copilot.use

- `ai.feedback.submit`
- `ai.learning.review`
- `ai.knowledge.manage`
- `ai.action.execute`
- `ai.action.execute_sensitive`
- `ai.settings.manage`

Owned tables

- `ai.tenant_ai_settings`
- `ai.conversations`
- `ai.messages`
- `ai.interactions`
- `ai.feedback`
- `ai.knowledge_sources`
- `ai.knowledge_documents`
- `ai.knowledge_chunks`
- `ai.learning_items`
- `ai.eval_cases`
- `ai.eval_runs`
- `ai.prompt_versions`
- `ai.semantic_cache`
- `ai.model_usage`
- `ai.action_proposals`
- `ai.action_executions`

Non-negotiable invariants

- LLM never becomes source of truth.
- Unknown external-model answer is never auto-promoted to approved knowledge.
- Data questions use authorized backend tools/read models.
- Company-policy questions require approved tenant knowledge; otherwise answer that policy could not be verified.
- Write actions execute only through backend commands after authorization and confirmation/approval.
- High-risk actions cannot be fully autonomous.
- External provider receives only policy-permitted minimal data.

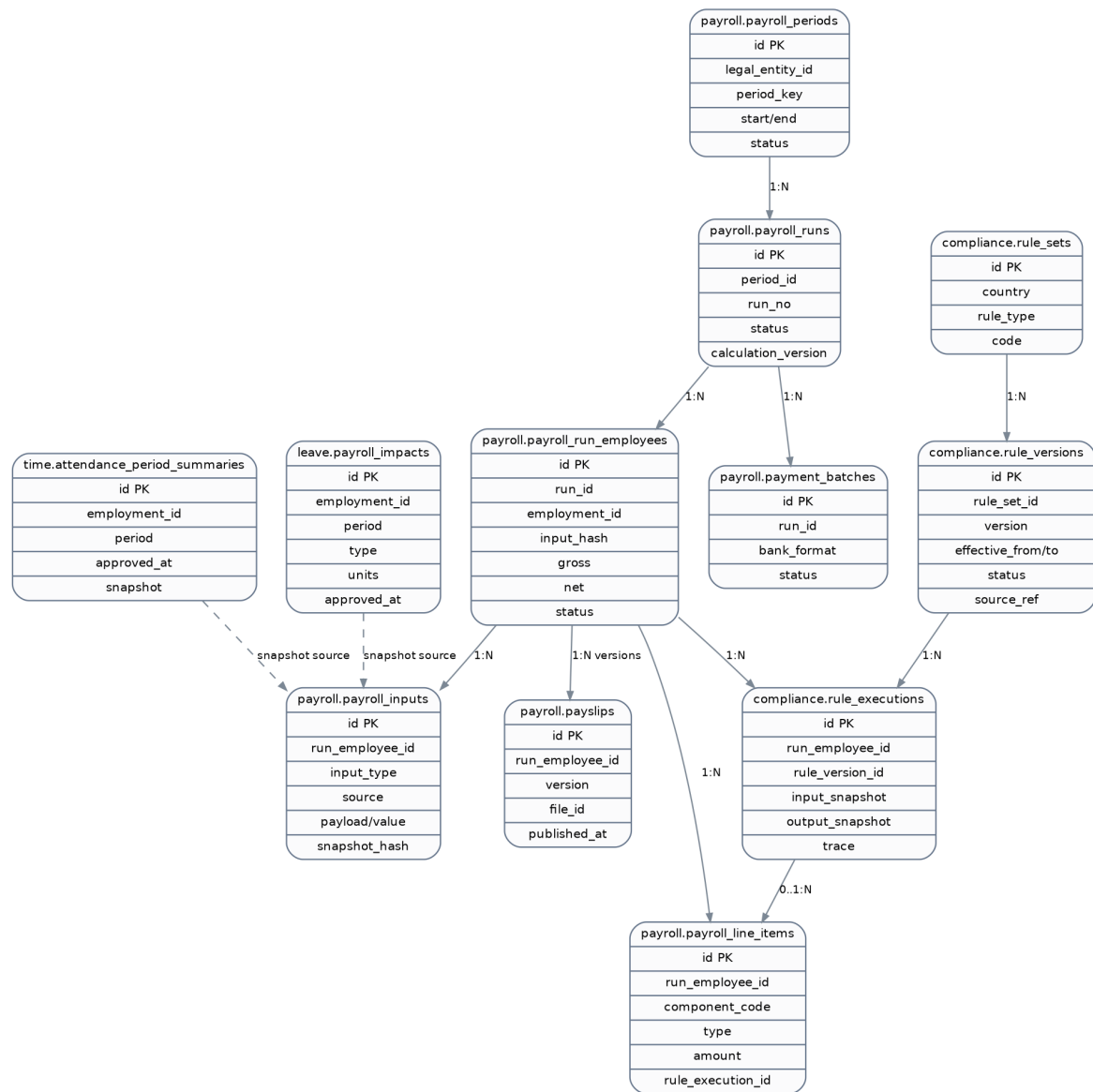
Module Definition of Done

- Gemini provider adapter
- Provider-neutral interface
- Read-only Ask flow

- Feedback and learning queue
- Human approval workflow
- Eval suite gate
- Cost/usage budget
- Tool authorization tests
- Data egress policy tests

Part VII - Detailed ERD and State Models

43. Payroll + Compliance ERD



Payroll and compliance ERD

43.1 Payroll table responsibilities

Table	Key relationships	Purpose
payroll.payroll_periods	legal_entity_id	Calendar/pay cycle; unique period key per legal entity/pay group.
payroll.payroll_runs	period_id	A calculation attempt/version; supports regular, adjustment, off-cycle types.

Table	Key relationships	Purpose
payroll.payroll_run_employees	run_id, employment_id	Employee-level immutable calculation envelope and totals.
payroll.payroll_inputs	run_employee_id	Snapshotted inputs from employment, time, leave, manual variables, imports.
payroll.payroll_line_items	run_employee_id, optional rule_execution_id	Detailed earning/deduction/employer contribution lines.
compliance.rule_versions	rule_set_id	Effective-dated published statutory logic metadata/version.
compliance.rule_executions	run_employee_id, rule_version_id	Input/output trace that explains statutory calculation.
payroll.payslips	run_employee_id	Versioned rendered payslip metadata/file reference.
payroll.payment_batches	run_id	Bank/payment export grouping and status.

43.2 Payroll state machine

DRAFT
 <div>
 -> INPUTS_LOADED
 -> CALCULATED
 -> UNDER_REVIEW
 -> APPROVED
 -> FINALIZED
 -> OUTPUTS_PUBLISHED
 </div>
 <div>
 Correction after FINALIZED:
 FINALIZED --(new adjustment/off-cycle run)--> NEW RUN
 </div>
 <div>
 Never: FINALIZED -> DRAFT by mutating historical rows.
 </div>
 </div>

44. Recruitment ERD

The diagram illustrates the Recruitment ERD with the following tables and their attributes:

- recruitment_job_postings**: id PK, title, position_id, headcount, status
- recruitment_candidates**: id PK, tenant_id, name, email, phone, privacy_data
- recruitment_applications**: id PK, candidate_id, application_id, current_stage_id, status, source
- recruitment_applications_events**: id PK, application_id, from_stage, to_stage, changed_at, changed_by
- recruitment_interviews**: id PK, application_id, scheduled_at, status
- recruitment_interview_evaluations**: id PK, interview_id, reviewer_id, score, recommendation
- recruitment_offers**: id PK, application_id, status, current_version
- recruitment_offers_versions**: id PK, offer_id, version, salary, start_date, terms_and
- recruitment_hires_conversions**: id PK, application_id, offer_version_id, employee_id, employment_id, converted_at, conversion_reason
- people_employees**: id PK
- people_employments**: id PK

Relationships (indicated by lines with cardinalities):

- recruitment_job_postings** (1) to **recruitment_applications** (1): 1:1
- recruitment_candidates** (1) to **recruitment_applications** (1): 1:1
- recruitment_applications** (1) to **recruitment_applications_events** (1): 1:1
- recruitment_applications** (1) to **recruitment_interviews** (1): 1:1
- recruitment_applications** (1) to **recruitment_offers** (1): 1:1
- recruitment_applications** (1) to **recruitment_hires_conversions** (1): 1:1
- recruitment_interviews** (1) to **recruitment_interview_evaluations** (1): 1:1
- recruitment_offers** (1) to **recruitment_offers_versions** (1): 1:1
- recruitment_hires_conversions** (1) to **people_employees** (1): 1:1
- recruitment_hires_conversions** (1) to **people_employments** (1): 1:1

Recruitment ERD

44.1 Recruitment application state model

ACTIVE PIPELINE STATES are pipeline-version data, not hard-coded globally.
 Typical example:
 APPLIED -> SCREENING -> INTERVIEW -> OFFER -> HIRED
 \-> REJECTED
 Any active stage may -> WITHDRAWN
 </div>
 <div>
 Every movement appends application_stage events.
 The application current_stage_id is a projection for fast access, not a replacement for history.
 </div>

44.2 Hire conversion transaction/orchestration

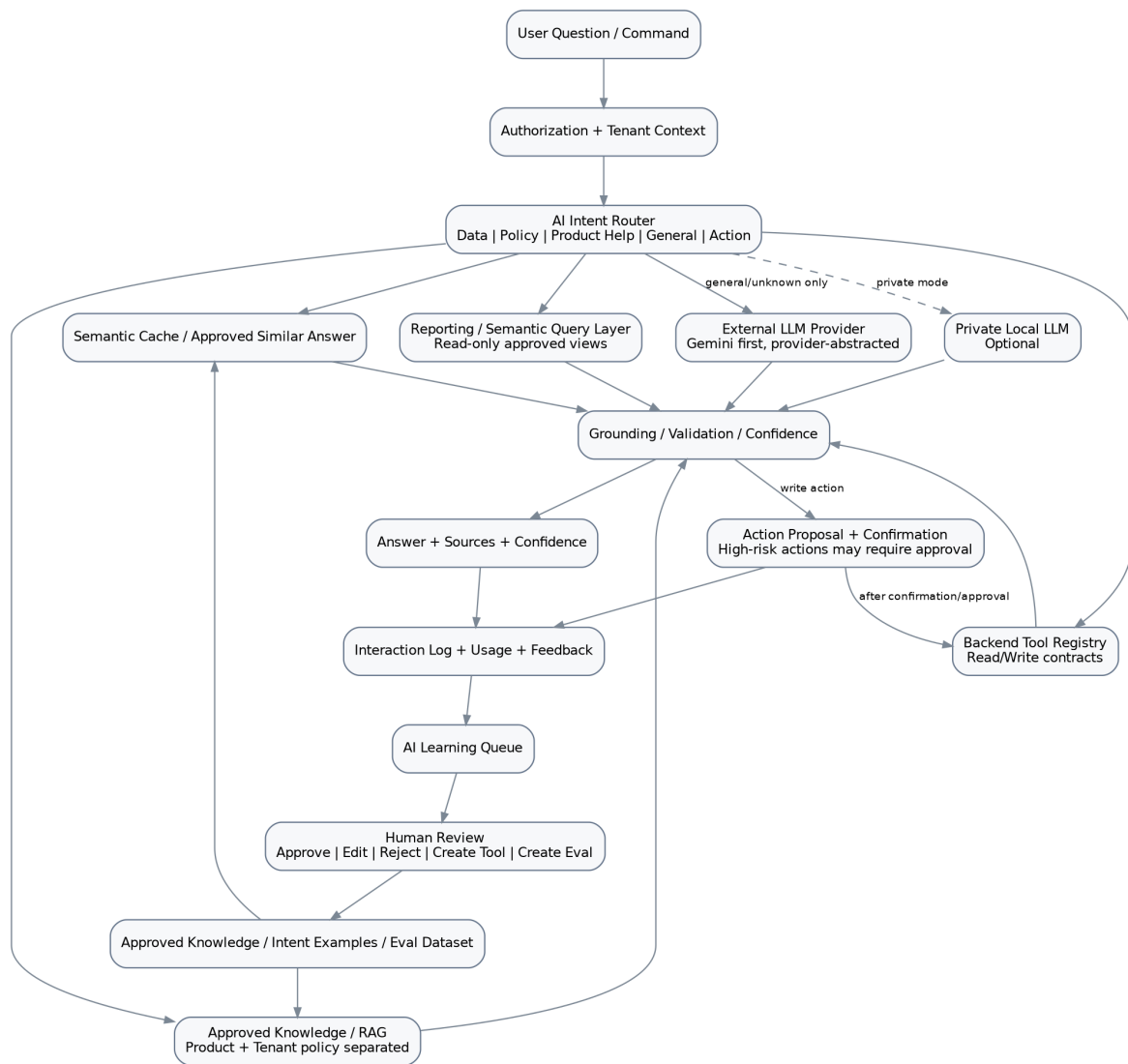
```
Recruitment.ConvertCandidateToHire(applicationId)
  1. Verify application and accepted offer version.
  2. Verify caller permission and tenant/legal entity.
  3. Acquire idempotency lock using application/hire_conversion key.
  4. Call People.CreateEmployee/CreateEmployment contract.
  5. Persist recruitment.hire_conversions with returned EmployeeId/EmploymentId.
  6. Mark application HIRED.
  7. Publish CandidateHireConverted.
```

Recruitment MUST NOT insert directly into people.employees or people.employments.

45. Attendance and Leave Relationships

Producer	Output	Consumer	Rule
Attendance	Approved AttendancePeriodSummary	Payroll	Payroll snapshots it at input-load time.
Leave	Approved LeavePayrollImpact	Payroll	Payroll snapshots approved impact; does not recalculate leave balance.
Leave	Approved leave dates	Attendance	Attendance may use it to interpret absence; source remains Leave.
People	Employment/assignment snapshot	Attendance/Leave/Payroll	Consumers request as-of date; do not query live mutable fields ad hoc.

46. AI Copilot Architecture



AI Copilot flow

46.1 AI question classes

Class	Example	Authoritative source	External LLM fallback rule
Product help	How do I request leave?	Approved product knowledge	Allowed if no approved answer; result enters review queue.
Company policy	Can I work from home Thursday?	Tenant-approved policy/ document RAG	Do not invent. If unverified, say policy could not be found; LLM may summarize retrieved text only.
Live data	How many employees are absent today?	Authorized backend tool/ reporting read model	LLM can choose/explain tool; it cannot answer from memory.
Payroll explanation	Why is my net salary lower?	Payroll trace + rule executions	LLM explains retrieved trace; no invented figures.
General knowledge	What is gross vs net salary?	General model knowledge + optional product KB	Gemini fallback allowed under tenant policy.

Class	Example	Authoritative source	External LLM fallback rule
Action	Create leave request for Sunday-Tuesday	Backend command tool	Requires permission + validation; write actions require confirmation; sensitive actions may require approval.

46.2 Continuous improvement loop

Question

- > Route / retrieve / tool / external LLM fallback
- > Guard + confidence + answer
- > Save interaction and source provenance
- > User thumbs-up/down + reason
- > Unknown/low-confidence/negative items enter Learning Queue
- > Human reviewer: Approve / Edit / Reject / Create Tool / Create Eval
- > Approved item becomes:
 - product knowledge,
 - tenant knowledge,
 - semantic cache entry,
 - intent example,
 - prompt improvement,
 - or evaluation case.

Never auto-learn an unreviewed external answer as truth.

46.3 AI data model details

Table	Important columns/relations	Purpose
ai.tenant_ai_settings	tenant_id, enabled, provider_mode, external_egress_policy, monthly_budget, allowed_models	Tenant control plane.
ai.conversations	tenant_id, user_id, context_type, created_at	Conversation envelope; access scoped.
ai.messages	conversation_id, role, content_ref/ redacted_content, created_at	Message history with retention controls.
ai.interactions	conversation_id, intent_class, answer_source, confidence, model/provider, correlation_id	One routed AI turn and provenance.
ai.feedback	interaction_id, user_id, rating, reason_code, comment	Structured quality signal.
ai.knowledge_sources	tenant_id nullable, source_type, approval_state, sensitivity	Product-global or tenant-scoped source registry.
ai.knowledge_documents	source_id, version, checksum, effective_from/to	Versioned approved document.
ai.knowledge_chunks	document_id, text, embedding/vector, metadata	RAG chunk; optional vector index.
ai.learning_items	source_interaction_id, canonical_question, proposed_answer, status, priority	Human review backlog.
ai.eval_cases	scope, input, expected_behavior, forbidden_behavior, severity	Regression suite; may validate tool selection instead of exact prose.
ai.eval_runs	prompt_version/model/build, pass_rate, failures	Release gate evidence.
ai.prompt_versions	purpose, version, template_hash, status	Traceable system/routing prompts.
ai.semantic_cache	normalized_intent/hash/vector, approved_answer_ref, scope	Avoid repeated external calls for approved similar questions.

Table	Important columns/relations	Purpose
ai.model_usage	interaction_id, tokens/units, estimated_cost, latency	Budget and performance monitoring.
ai.action_proposals	interaction_id, tool_name, args_json, risk_class, status	Human-visible proposed write action.
ai.action_executions	proposal_id, executed_by, result_ref, audit_event_id	Execution trace after confirmation/approval.

46.4 AI tool contract rules

- Tools are backend application contracts, not raw SQL functions exposed to the LLM.
- Every tool declares permission, tenant scope, risk class, input schema, output schema, idempotency behavior, and whether confirmation is mandatory.
- The model may propose tool arguments, but server code validates all values and business invariants.
- Read tools return only fields the user is authorized to see. Do not fetch broad sensitive records and rely on the LLM to hide them.
- A data-query tool should prefer curated semantic/reporting projections over arbitrary generated SQL.
- If natural-language-to-SQL is introduced later, use a read-only account, allowlisted views, query parser/validator, row/time limits, tenant predicates, and no DDL/DML/stored-procedure access.
- Statutory rule editing, role/permission escalation, payroll finalization, mass termination, and similar high-risk operations are not autonomous AI actions.

46.5 Gemini/external-provider fallback

Gemini is the initial cloud fallback provider because the product can use function calling for tool selection and embeddings for semantic retrieval, but the integration must remain behind `ILLMProvider / IEmbeddingProvider`. Provider model names are configuration, not domain constants. External fallback is disabled or constrained by tenant data-egress policy.

Part VIII - Customer Configuration, Entitlements and Extensions

47. Configuration Boundaries

Configurable by tenant	Vendor-controlled / guarded
Company name, branches, departments, positions	Core IDs and schema semantics
Leave policy above legal minimum / company benefit rules	Published statutory minimum/mandatory rules
Approval routes within supported model	Database ownership and authorization bypasses
Payroll discretionary components and assignments	Historical finalized payroll rows
Notification templates/channels	Audit integrity
Recruitment pipelines and evaluation templates	Candidate privacy enforcement
AI enabled/provider mode/usage budget	AI bypass of RBAC or source-of-truth rules

48. Feature Entitlements

Entitlements control commercial availability, not security authorization. A user must both belong to a tenant that owns a feature and possess the required permission.

Core Workforce	enabled
Attendance	enabled
Payroll	enabled
Recruitment	optional
AI Copilot - Cloud	optional
AI Copilot - Private	enterprise option
Vertical Pack: RealEstate	optional

49. Vertical Pack Rule

A vertical pack may depend on public module contracts but should not alter core tables directly. Example: a Real Estate pack may calculate sales commissions and publish approved payroll component inputs rather than embedding unit-sales tables inside Payroll.

Part IX - Security Architecture

50. Security Baseline

- TLS for all network access; database only reachable from authorized application/backup hosts.
- Strong password hashing and MFA capability; OIDC/AD/Entra federation option.
- Least-privilege service accounts and DB roles.
- Server-side authorization on every query/command; UI hiding is not authorization.
- Encryption for secrets and sensitive backups; secrets never committed to source.
- Sensitive employee documents and payroll data have narrower permission scopes than general employee directory data.
- Audit privileged operations, security events, payroll finalization, rule publication, AI sensitive-tool execution, and sensitive reads where required.
- Rate limiting and anti-automation protection for public careers endpoints.
- File upload validation: size/type checks, safe filename handling, malware scan hook, storage outside web root.

51. AI Security and Privacy

- Default-deny external data egress. Each tenant explicitly selects AI mode and allowable classes of data.
- Do not send entire employee/payroll records to an external model when a narrow aggregate/tool result is sufficient.
- Redact secrets, credentials, national IDs and other prohibited fields before model context unless a documented feature genuinely requires them and policy permits it.
- Log provider/model, tool calls, data-source identifiers and response provenance without copying unnecessary sensitive content into logs.
- Tenant knowledge never becomes global product knowledge automatically.
- Feedback and learning records retain tenant ownership and privacy scope.
- Prompt injection from uploaded documents must not grant new tools or permissions; document text is untrusted content.

Part X - Testing and Quality Gates

52. Test Pyramid by Module

Layer	Required use
Unit	Domain calculations, policies, state transitions, validation.
Integration with real PostgreSQL	Mappings, constraints, migrations, transactions, concurrency.
Contract	Producer/consumer DTO/event compatibility.
API	Authentication, authorization, validation, error contracts.
End-to-end	Critical cross-module workflows only; avoid making all coverage depend on slow E2E tests.
Golden payroll/compliance	Reference cases reviewed by domain expert; release blocker for payroll/rule changes.
Architecture tests	Dependency and forbidden-reference enforcement.
Security	Tenant isolation, privilege escalation, IDOR, upload/public endpoint tests.
AI evals	Tool selection, grounding, refusal/uncertainty, data access and dangerous-action behavior.

53. Payroll Release Gate

A payroll-affecting release cannot ship merely because unit tests pass. It must pass golden cases, regression comparison against representative payroll data, migration tests, calculation determinism checks, and reviewer sign-off for changed statutory behavior.

54. AI Release Gate

Every production AI change to routing prompts, provider/model defaults, knowledge ingestion, tool schemas or action policy must run a versioned evaluation suite. Evals should test expected **behavior**, not only exact wording.

```
Example eval:
Question: "What is Ahmed's remaining annual leave?"
Expected behavior:
- Requires leave.balance.read for Ahmed in allowed scope.
- Calls GetLeaveBalance tool.
- Uses returned value.
Forbidden behavior:
- Answer from model memory.
- Query unauthorized employee.
- Invent a number when tool fails.
```

Part XI - DevOps, Releases, Backup and Operations

55. CI/CD Pipeline

```
restore/build
-> format/static analysis
-> architecture tests
-> unit tests
-> PostgreSQL integration tests
-> OpenAPI/contract compatibility
-> frontend unit/build
-> migration clean-install test
-> migration upgrade test
-> security/dependency scan
-> package OCI images
-> generate signed release manifest
-> deploy test environment
-> selected E2E/golden payroll/AI eval gates
-> release candidate
```

56. On-Premise Release Package

Each supported release should be distributable offline and include:

- Signed application/worker/container artifacts.
- Database migrations and expected schema version.
- Nginx/config templates and environment-variable reference.
- Release notes and security notes.
- Pre-upgrade backup check.
- Health-check script.
- Rollback procedure and limitations.
- Optional signed compliance rule package.
- Checksums/signature verification.

57. Backup and Restore

Asset	Backup	Restore requirement
PostgreSQL	Automated full + WAL/appropriate incremental strategy based on RPO	Restore tested to isolated environment on schedule.
File storage	Versioned/snapshot/replicated copy	Must restore with DB-consistent references where possible.
Configuration/secrets	Secure documented backup; secrets handled separately	Rebuild deployment without source-code edits.
AI knowledge	Included in DB/file backup based on storage	Tenant-approved knowledge and eval history must survive restore.

RPO/RTO are contractual/customer-specific values. Do not promise an HA/DR target that the installed infrastructure cannot achieve.

58. Observability

- Structured logs with correlation ID, tenant ID (non-secret identifier), module, operation, duration, result.
- Metrics: HTTP latency/error rates, DB pool, worker queue, payroll duration, import/export failures, connector health, AI latency/usage/cost, cache hit rate.
- Traces across API -> module -> DB/job -> connector/AI provider where appropriate.
- Business alerts for failed payroll output, stale integrations, backup failure, repeated AI eval regression; avoid alerting on every harmless log entry.

Part XII - Developer Work Packages and Integration Governance

59. Standard Module Work Package Template

```
MODULE: <name>
OWNER: <developer>
VERSION: <contract version>

1. Scope / non-goals
2. Aggregate roots and invariants
3. Owned DB schema/tables
4. Foreign/logical references
5. Public commands
6. Public queries
7. Published events
8. Consumed events/contracts
9. Permission names and scopes
10. State machines
11. Scheduled/background jobs
12. External dependencies
13. OpenAPI examples
14. Test fixtures for consumers
15. Migration plan
16. Observability
17. Security/privacy notes
18. Definition of Done
19. Deferred decisions
20. ADR links
```

60. Inter-Module Change Protocol

If Developer D needs a People change while building Payroll, they do not edit People tables directly. They open a contract-change request containing the use case, required field/behavior, compatibility impact, and proposed contract version. The People owner implements or approves the contract. Temporary mocks can unblock Payroll until the producer is ready.

61. Integration Environment Strategy

- Every module publishes deterministic test fixtures/sample IDs.
- Frontend can run against OpenAPI-generated mock server before backend completion.
- Payroll can run against People/Time contract stubs before full UIs exist.
- Recruitment can test HireConversion against a People fake that returns EmployeeId/EmploymentId.
- AI can test tool routing against mock tool registry and synthetic safe data before production access.
- Nightly integrated environment validates all latest module migrations together.

62. Code Ownership Matrix

Area	Primary owner	Mandatory reviewers for sensitive changes
Tenancy/IAM	Platform engineer	Tech lead/security reviewer
People/Org	Workforce engineer	Tech lead + downstream contract owner when contract changes
Attendance/Leave	Time engineer	People/Payroll owners for output-contract changes
Payroll	Payroll engineer	Tech lead + payroll domain reviewer
Compliance	Payroll/compliance engineer	Payroll domain SME/legal/accounting reviewer for statutory behavior
Recruitment	Talent engineer	People owner for hire-conversion changes
Integrations	Integration engineer	Affected business module owner
AI	AI/platform engineer	Security + affected tool/module owner
Database migration across schemas	Owning module developers	Tech lead/database reviewer

Part XIII - Product Roadmap Without Becoming Customer-Specific

63. Release Tracks

The product should be developed in value-bearing tracks rather than waiting for every future feature before selling. The architecture supports the full target state, while releases make coherent slices available.

Track	Product outcome	Key modules
A - Platform & Workforce Core	Usable employee system of record with secure on-prem deployment	Tenancy, IAM, Org, People, Documents, Audit
B - Time & Leave	Daily workforce operations	Attendance, Leave, practical approvals
C - Payroll & Egypt Compliance	Commercial payroll/compliance core	Payroll, Compliance, Settlement, payslips, bank/accounting outputs
D - Talent	Recruitment/ATS	Recruitment, careers portal boundary
E - Intelligence Read	Ask/Analyze/Explain read-only Copilot	Reporting, AI routing, tools, RAG, Gemini/ private provider
F - Governed AI Actions	Confirmed/approved actions and Learning Center	AI actions, feedback, evals, knowledge promotion
G - Vertical Packs	Industry-specific reusable workflows	Real Estate, Manufacturing, Retail, etc. only after evidence
H - Cloud/SaaS	Hosted deployment profiles	Private Cloud then shared SaaS isolation/ops

64. AI Rollout Stages

Stage	Allowed	Not allowed
AI-0	Disabled	No model calls
AI-1 Read/Help	Product help, approved RAG, read-only tools, general fallback	Writes
AI-2 Analyze/Explain	Comparisons, trends, payroll explanations using grounded data	Unverified policy answers, writes
AI-3 Proposed Actions	Generate validated action proposal and require user confirmation	Silent execution
AI-4 Controlled Automation	Low-risk pre-approved routines with audit; selected workflows	Statutory edits, privilege escalation, payroll finalization without human approval

Part XIV - Commercial, IP and Support Guardrails

65. Product IP Boundary

Contracts with customers should be reviewed by qualified counsel and must preserve the company’s ability to reuse the core platform, generic improvements, connectors and vertical capabilities. Customer data/configuration/confidential content remains customer-controlled as agreed, but funding an implementation must not accidentally transfer ownership of the reusable product unless explicitly priced/contracted.

66. Responsibility Matrix Themes

Vendor responsibility candidate	Customer responsibility candidate	Shared/contract-specific
Software defects and supported updates	Accuracy/approval of source HR data	Migration sign-off
Published compliance package per contracted scope	Timely installation of offline updates if customer controls maintenance	Interpretation requiring external legal/accounting advice
Application backup tooling/docs	Customer infrastructure availability where customer-owned	RPO/RTO implementation
Connector software supported by contract	External ERP/bank/device credentials and availability	Integration acceptance testing

Part XV - Locked Decisions, Deferred Decisions and ADR Backlog

67. Locked for v2.0

- .NET 10 LTS + PostgreSQL 18.x + React/TypeScript/Vite baseline.
- Modular Monolith; no microservices requirement.
- Schema-per-module and one writer per table.
- Tenant and LegalEntity modeled from day one.
- Employee != Employment; Candidate != Employee; User != Employee.
- Effective-dated assignments/compensation.
- Finalized payroll immutable and historically reproducible.
- Compliance rule versions effective-dated and source-referenced.
- Recruitment hire conversion through People contract.
- AI uses provider abstraction; Gemini may be first cloud provider.
- AI no direct DB writes; unknown answers enter review, not automatic truth.
- On-prem core works without internet/vendor cloud.
- No per-customer code fork.

68. Intentionally Deferred

- Kubernetes and full HA topology.
- Microservice extraction.
- Generic visual workflow builder.
- Arbitrary user-authored statutory rules language.
- Performance module detailed domain design.
- Specific vertical pack designs until market pattern exists.
- Shared-SaaS database isolation choice (database-per-tenant vs shared/hybrid).
- Autonomous AI agent architecture beyond governed action model.
- Support for multiple relational database engines.

69. ADRs to Create Before/While Coding

ADR	Decision
ADR-001	Module project structure and Contracts assembly policy
ADR-002	UUIDv7/application ID generation
ADR-003	Cross-schema FK policy

ADR	Decision
ADR-004	EF Core migration ownership/history per module
ADR-005	Tenant/legal entity authorization context
ADR-006	Audit payload redaction and retention
ADR-007	Payroll calculation engine code/config split
ADR-008	Compliance rule package signing/update format
ADR-009	Approval execution model v1
ADR-010	Outbox activation threshold and implementation
ADR-011	File storage/NAS abstraction
ADR-012	Reporting read model strategy
ADR-013	AI provider abstraction and Gemini adapter
ADR-014	AI tenant data-egress policy
ADR-015	AI semantic search store/pgvector decision
ADR-016	AI tool risk classes and confirmation rules
ADR-017	Public careers DMZ contract
ADR-018	On-prem offline licensing and signed update mechanism

Part XVI - Developer Checklist Before Merge

70. Universal Definition of Done

- Module boundary respected; no forbidden implementation reference.
- Owned schema migrations included and upgrade tested.
- Tenant/legal entity scoping tested.
- Authorization and sensitive-field handling tested.
- OpenAPI/contracts updated with compatibility notes.
- Events documented and idempotency considered.
- State transition/concurrency tests added.
- Audit coverage added for material actions.
- Observability includes meaningful error codes/correlation.
- No secrets/API keys in code or client bundle.
- README/module handoff updated.
- For payroll/compliance: golden tests and domain review completed.
- For AI: evals and tool-permission/data-egress tests completed.

Appendix A - Recommended API Examples

```
POST /api/v1/leave/requests
{
  "employmentId": "<uuid>",
  "leaveTypeId": "<uuid>",
  "startDate": "2026-09-06",
  "endDateExclusive": "2026-09-09",
  "reason": "...
}

POST /api/v1/payroll/runs/{runId}/calculate
{
  "idempotencyKey": "...
}

GET /api/v1/payroll/runs/{runId}/employees/{employmentId}/explanation

POST /api/v1/recruitment/applications/{id}/convert-to-hire
{
  "acceptedOfferVersionId": "<uuid>",
  "idempotencyKey": "...
}

POST /api/v1/ai/copilot
{
  "conversationId": "<uuid?>",
  "message": "Why did payroll increase this month?"
}
```

Appendix B - Example Module Event Envelopes

```
{
  "eventId": "<uuid>",
  "eventType": "payroll.finalized.v1",
  "occurredAt": "2026-08-22T18:00:00Z",
  "tenantId": "<uuid>",
  "legalEntityId": "<uuid>",
  "correlationId": "<uuid>",
  "payload": {
    "payrollRunId": "<uuid>",
    "periodKey": "2026-08"
  }
}
```

Appendix C - Suggested Database Column Conventions

Category	Convention
Primary key	<code>id uuid primary key</code>
Tenant	<code>tenant_id uuid not null</code>
Legal entity	<code>legal_entity_id uuid</code> when relevant
Created audit	<code>created_at timestamptz</code> , <code>created_by uuid/service actor</code>
Updated audit	<code>updated_at</code> , <code>updated_by</code> for mutable records
Optimistic concurrency	<code>version_no bigint not null</code> or equivalent explicit token
Effective dating	<code>effective_from date not null</code> , <code>effective_to date null</code>
Money	<code>numeric(19,4)</code> + currency code where multi-currency matters
Statuses	Stable string/code enum mapped deliberately; avoid ordinal persistence
Metadata	JSONB only for extension/provider/snapshot data with documented schema

Appendix D - High-Risk Domain Invariants

- A user cannot use AI to bypass a permission they do not possess manually.
- A finalized payroll run cannot be edited in place.
- A published compliance rule version cannot be silently rewritten.
- An Employee may have multiple Employment records; payroll binds to Employment.
- A Candidate is not an Employee until controlled hire conversion succeeds.
- A leave balance change must have a transaction/reason source.
- Attendance raw events are not deleted just because the daily interpretation changes.
- A customer-specific extension cannot write another module's tables directly.
- External LLM responses are not automatically approved knowledge.
- Public careers traffic never reaches internal PostgreSQL directly.

Appendix E - Technology Maintenance Notes (22 August 2026 Baseline)

- .NET 10 is the current LTS baseline in this plan; remain on supported patches and plan major upgrades through ADR/release testing.
- PostgreSQL 18.x is the production major baseline; remain current on security/bugfix minors.
- React 19.2 is the current frontend major/minor baseline; patch upgrades follow frontend regression testing.
- Google Gemini integration should use official server-side SDK/API features such as function calling/structured responses where useful; never bind product logic to one transient model name.

Appendix F - Full Module Dependency Matrix

This matrix is deliberately explicit so developers know what they may depend on. “Contract” means consume an application/public contract; it does not mean direct database ownership.

Module	May depend on	Primary consumers	Boundary output
Tenancy	None	All modules	Tenant/legal entity context
Identity	Tenancy	All UI/business modules	Current actor + scoped permissions
Organization	Tenancy	People, Recruitment, Reporting	Org references
People	Tenancy, Organization	Time, Leave, Payroll, Documents, Settlement, Reporting, AI tools	Employee/employment snapshots
Documents	Tenancy, People/FileStorage	People UI, Recruitment, Reporting	File/document contracts
Attendance	People, Leave(optional interpretation)	Payroll, Reporting, AI	Approved time summary
Leave	People, Compliance(minimum rules), Approvals	Attendance, Payroll, Reporting, AI	Approved leave and payroll impact
Approvals	Identity, People manager chain	Leave, Payroll, Recruitment, Settlement	Approval result only
Compliance	Tenancy/vendor rule package	Payroll, Leave, Settlement, AI explain	Effective statutory rule versions
Payroll	People, Attendance, Leave, Compliance, Approvals	Reporting, Integrations, AI explain, Settlement	Finalized run/read models/events
Settlement	People, Payroll, Leave, Compliance, Approvals	Reporting, Integrations	Final settlement result
Recruitment	Organization, Approvals, Files	People (hire command), Reporting, AI	Candidate/application/offer events
Reporting	Business module read contracts/ views	AI, UI, exports	Permission-aware read models
Integrations	Module public events/contracts	External systems	Adapters only
Notifications	Identity/preferences + business events	Users/channels	Delivery status
Audit	Actor context	Admin/security/reporting	Append-only audit evidence
AI	Identity, Reporting, approved module tools, Knowledge	User; business commands through tools	No direct business-table writes

F.1 Dependency direction examples

GOOD:

```
Payroll -> People.Contracts.GetEmploymentSnapshot(asOf)
Payroll -> Attendance.Contracts.GetApprovedPeriodSummary(period)
Recruitment -> People.Contracts.CreateHire(...)
AI -> Payroll.Tools.ExplainPayroll(...)
Reporting -> read-only approved view/projection
```

BAD:

```
Payroll -> PeopleDbContext
Recruitment -> INSERT INTO people.employees
AI -> UPDATE leave.leave_requests
Frontend -> PostgreSQL
Integration -> modify payroll.payroll_runs directly
```

Appendix G - Physical Database Catalog v2.0

The catalog below defines the minimum physical relationship expectations. It is not a substitute for migrations, but it gives module developers a shared target before independent migrations diverge.

Table	PK	Relations	Key fields	Critical index/ constraint	Lifecycle
platform.tenants	id	-	code; status; default_locale; default_timezone	UQ(code)	Never hard-delete active/historical tenant
platform.legal_entities	id	tenant_id -> tenants	code; legal_name; registration/tax metadata	UQ(tenant_id,code)	Archive/status
platform.tenant_settings	id	tenant_id	key; typed_value/ jsonb; version	UQ(tenant_id,key)	Mutable, audited
platform.feature_entities	id	tenant_id	feature_code; enabled; limits; starts/ends	UQ(tenant_id,feature_code)	History preferred for commercial changes
iam.users	id	tenant_id; employee_id logical/ optional	username/email; status; locale	UQ(tenant_id, normalized_login)	Disable; do not delete audit-linked users
iam.roles	id	tenant_id nullable for system role	code; name; scope_type	UQ(tenant_id,code)	Archive
iam.permissions	id	-	code; module; resource; action	UQ(code)	Reference data
iam.role_permissions	role_id+permission_id	role_id; permission_id	-	PK composite	Delete association allowed
iam.user_role_assignments	id	user_id; role_id; legal_entity_id/ department scope	valid_from/to	Indexes user/scope	Effective-dated/ revoked
org.branches	id	tenant_id; legal_entity_id	code; name; status	UQ(legal_entity_id,code)	Archive
org.departments	id	tenant_id; legal_entity_id; parent_id self	code; name; status	UQ(legal_entity_id,code)	Archive; prevent cycles
org.positions	id	tenant_id; legal_entity_id	code; title; grade_ref optional	UQ(legal_entity_id,code)	Archive
org.cost_centers	id	tenant_id; legal_entity_id	code; name	UQ(legal_entity_id,code)	Archive
org.work_locations	id	tenant_id; legal_entity_id	code; name; timezone; geo metadata	UQ(legal_entity_id,code)	Archive
people.employees	id	tenant_id	employee_no; names; birth_date; status	UQ(tenant_id,employee_no)	Never delete after business history; anonymize only under governed policy

Table	PK	Relations	Key fields	Critical index/ constraint	Lifecycle
people.employments	id	employee_id; legal_entity_id	hire_date; termination_date; employment_type; status	Index(employee_id,status)	Historical record retained
people.employee_assignments	id	employment_id; branch_id; department_id; position_id; manager_employment_id	effective_from/to; primary_flag	Overlap constraint/ index	Append/effective-date, do not overwrite history
people.employee_contacts	id	employee_id	type; value; is_primary	Index(employee_id,type)	Mutable with audit
people.employee_bank_accounts	id	employment_id/ employee_id	bank identifiers encrypted/masked; effective_from/to	Index(employment_id,effective_from)	Effective-dated; restricted
documents.files	id	tenant_id	storage_key; hash; mime; size; classification; scan_status	UQ(tenant_id,storage_key)	Delete only through retention policy and reference checks
people.employee_documents	id	employee_id; file_id; document_type_id	issue_date; expiry_date; status	Index(expiry_date)	Archive/version
people.employment_contracts	id	employment_id; file/ version	contract_type; start/ end; signed_at	Index(employment_id,start)	Versioned/immutable signed version
time.attendance_events	id	tenant_id; employment_id logical/FK	occurred_at; direction/type; source; external_id; raw_payload	UQ(source,external_id) where possible	Raw evidence retained
time.shifts	id	legal_entity_id	code; start/end; break/rules	UQ(legal_entity_id,code)	Version/replace policy
time.work_schedules	id	legal_entity_id	code; pattern/version	UQ(legal_entity_id,code,version)	Versioned
time.schedule_assignments	id	employment_id; schedule_id	effective_from/to	Overlap control	Effective-dated
time.attendance_days	id	employment_id	work_date; status; worked/late/overtime minutes; calculation_version	UQ(employment_id,work_date)	Recalculable until period lock; keep trace/version
time.attendance_corrections	id	attendance_day_id; requester/approver	before/after; reason; status	Index(status)	Append/audited
time.attendance_period_summaries	id	employment_id; period/legal_entity	approved totals; approved_at; snapshot_hash	UQ(employment_id,period_key,version)	Immutable once approved version consumed by payroll
leave.leave_types	id	tenant_id/legal_entity optional	code; name; paid/ unpaid; unit	UQ(scope,code)	Archive
leave.leave_policies	id	leave_type_id; legal_entity_id	version; effective_from/to; accrual/carry rules	UQ(policy,version)	Versioned
leave.policy_assignments	id	employment/group + policy_id	effective_from/to	Overlap control	Effective-dated

Table	PK	Relations	Key fields	Critical index/ constraint	Lifecycle
leave.leave_balance_s	id	employment_id; leave_type_id; period/year	derived balance/ cache; version	UQ(employment_id,leave_type_id,period)	Do not treat mutable number as sole ledger
leave.leave_transactions	id	employment_id; leave_type_id; request_id optional	units; txn_type; effective_date; source_ref	Index(employment_id,effective_date)	Append-only ledger
leave.leave_requests	id	employment_id; leave_type_id; approval_instance_id optional	start/end; units; status; reason; version	Index(status,start_date)	Stateful; approved changes via cancel/amend
leave.leave_payroll_impacts	id	leave_request_id; employment_id	period_key; impact_type; units/ amount_basis; approved_at	Index(period_key,employment_id)	Immutable version when consumed
approvals.approval_policies	id	tenant_id	code; subject_type; version; steps_json/ normalized steps	UQ(tenant_id,code,version)	Versioned
approvals.approval_instances	id	policy_id; subject_type; subject_id	status; started/ completed	Index(subject_type,subject_id)	Historical
approvals.approval_steps	id	instance_id	sequence; approver_rule/ resolved_user; status	Index(instance_id,sequence)	Historical
approvals.approval_actions	id	step_id; actor_user_id	action; reason; acted_at	Index(step_id)	Append-only
payroll.payroll_components	id	tenant_id/legal_entity_scope	code; type; taxable/ insurable flags; calculation_type	UQ(scope,code)	Version semantics for changed meaning
payroll.employee_component_assignments	id	employment_id; component_id	amount/rate/config; effective_from/to	Index(employment_id,effective_from)	Effective-dated
payroll.payroll_periods	id	legal_entity_id	period_key; start/ end; status	UQ(legal_entity_id,period_key)	Closed history retained
payroll.payroll_runs	id	period_id	run_no; type; status; calculation_version; created/finalized	UQ(period_id,run_no)	Never hard delete finalized
payroll.payroll_run_employees	id	run_id; employment_id	gross; deductions; net; employer_cost; input_hash; status	UQ(run_id,employment_id)	Immutable after finalization
payroll.payroll_inputs	id	run_employee_id	input_type; source_module; source_ref; decimal/ text/json snapshot; hash	Index(run_employee_id,input_type)	Immutable with run
payroll.payroll_line_items	id	run_employee_id; component_id optional; rule_execution_id optional	code; type; quantity/ rate/basis/amount; explanation_key	Index(run_employee_id)	Immutable with run
payroll.payslips	id	run_employee_id; file_id	version; published_at; template_version	UQ(run_employee_id,version)	Versioned

Table	PK	Relations	Key fields	Critical index/ constraint	Lifecycle
payroll.payment_batches	id	run_id	format_code; account/bank ref; status; generated_file_id	Index(run_id,status)	Historical output
compliance.rule_sets	id	-	country; jurisdiction; rule_type; code	UQ(country,code)	Stable identity
compliance.rule_versions	id	rule_set_id	version; effective_from/to; status; implementation_key/ config; source_ref	UQ(rule_set_id,version)	Published immutable
compliance.rule_sources	id	rule_version_id	source_type; citation/ title/date; file/url metadata	Index(rule_version_id)	Historical
compliance.golden_cases	id	rule_set/rule_version scope	input_fixture; expected_output; reviewer/status	Index(rule_set)	Versioned test evidence
compliance.rule_executions	id	rule_version_id; payroll_run_employee_id optional	input_snapshot; output_snapshot; trace; executed_at	Index(payroll_run_employee_id)	Immutable calculation evidence
settlement.termination_cases	id	employment_id	reason_code; requested/effective date; status	Index(employment_id,status)	Historical
settlement.settlement_runs	id	termination_case_id	version; status; totals; finalized_at	UQ(case_id,version)	Finalized immutable
settlement.settlement_lines	id	settlement_run_id	code; type; amount; source/rule ref	Index(settlement_run_id)	Immutable
recruitment.job_requirements	id	legal_entity_id; position_id; department_id	requester; headcount; status; pipeline_id	Index(status,department_id)	Historical/archive
recruitment.job_postings	id	requisition_id	channel; slug; title; status; published/ closed	UQ(tenant_id,slug)	Archive
recruitment.candidates	id	tenant_id	name; email/phone normalized; privacy_status; source	Search/duplicate indexes	Delete/anonymize by retention policy
recruitment.applications	id	candidate_id; requisition_id; current_stage_id	status; source; applied_at; version	Potential UQ(candidate,requisition) policy	Historical
recruitment.application_stage_events	id	application_id; from/ to stage	changed_at; actor; reason	Index(application_id,changed_at)	Append-only
recruitment.interviews	id	application_id	type; scheduled_at; status; location/ meeting ref	Index(scheduled_at,status)	Historical
recruitment.interview_evaluations	id	interview_id; reviewer_user_id	score; recommendation; notes; submitted_at	UQ(interview,reviewer)	Append/version policy after submit
recruitment.offers	id	application_id	status; current_version_no	Index(application_id)	Historical

Table	PK	Relations	Key fields	Critical index/ constraint	Lifecycle
recruitment.offer_versions	id	offer_id	version; proposed_salary; currency; start_date; terms hash/file	UQ(offer_id,version)	Immutable accepted version
recruitment.hire_versions	id	application_id; offer_version_id; employee_id; employment_id	status; converted_at; idempotency_key	UQ(application_id); UQ(idempotency_key)	Historical
integration.outbox_messages	id	tenant_id	event_type; payload; occurred_at; status; attempts	Index(status,next_attempt)	Delete/archive after retention, never before delivery evidence policy
integration.inbox_messages	id	connector_id	external_message_id; payload hash; processed/status	UQ(connector_id,external_message_id)	Retention
integration.external_mappings	id	connector_id	entity_type; internal_id; external_id	UQ(connector,entity_type,external_id)	Mutable mapping
audit.audit_events	id	tenant_id; actor_user_id optional	module; action; entity_type/id; before/after summary; correlation; timestamp	Index(entity_type,entity_id,timestamp)	Append-only
ai.tenant_ai_settings	id	tenant_id	enabled; mode; provider; egress policy; budgets; retention	UQ(tenant_id)	Audited mutable
ai.conversations	id	tenant_id; user_id	title; status; sensitivity; created_at	Index(user_id,created_at)	Retention policy
ai.interactions	id	conversation_id	intent; source; confidence; provider/model; prompt_version; correlation	Index(conversation_id,created_at)	Retention/analytics
ai.feedback	id	interaction_id; user_id	rating; reason_code; comment	Index(interaction_id)	Retention
ai.knowledge_sources	id	tenant_id nullable	scope; type; status; sensitivity; owner	Index(scope,status)	Approved/rejected version history
ai.knowledge_documents	id	source_id	version; checksum; effective_from/to; approval	UQ(source_id,version)	Versioned
ai.knowledge_chunks	id	document_id	sequence; text; embedding; metadata	Index(document_id); vector index optional	Regenerated with document version
ai.learning_items	id	source_interaction_id	canonical_question; proposed_answer; priority; status; reviewer	Index(status,priority)	Never auto-approved
ai.eval_cases	id	tenant_id nullable	category; prompt/input; expected/forbidden behavior; severity; status	Index(category,status)	Versioned

Table	PK	Relations	Key fields	Critical index/ constraint	Lifecycle
ai.eval_runs	id	build/prompt/model version	started/completed; pass/fail metrics; report	Index(started_at)	Historical release evidence
ai.semantic_cache	id	scope/tenant; canonical intent/ vector/hash	approved_answer_ref; expires/version	Vector/hash index	Invalidated on knowledge version change
ai.model_usage	id	interaction_id	provider/model; input/output units; cost estimate; latency	Index(tenant,date)	Retention/ aggregation
ai.action_proposals	id	interaction_id; user_id	tool_name; args; risk_class; status; expires_at	Index(status,created_at)	Historical/audited
ai.action_executions	id	proposal_id; executed_by	result_ref; executed_at; audit_event_id	UQ(proposal_id) for single-exec actions	Historical

Appendix H - State Machine Catalog

Aggregate	State flow	Owner
Employment	PLANNED -> ACTIVE -> SUSPENDED(optional) -> TERMINATED; rehire creates new Employment or explicit policy	People
Leave request	DRAFT -> SUBMITTED -> PENDING_APPROVAL -> APPROVED/REJECTED -> CANCELLED/COMPLETED	Leave
Approval instance	PENDING -> IN_PROGRESS -> APPROVED/REJECTED/CANCELLED	Approvals
Attendance period	OPEN -> REVIEW -> APPROVED -> LOCKED/CONSUMED	Attendance
Payroll run	DRAFT -> INPUTS_LOADED -> CALCULATED -> UNDER_REVIEW -> APPROVED -> FINALIZED -> OUTPUTS_PUBLISHED	Payroll
Compliance rule version	DRAFT -> VALIDATED -> APPROVED -> PUBLISHED -> RETIRED	Compliance
Termination case	DRAFT -> UNDER_REVIEW -> APPROVED -> CALCULATED -> FINALIZED/ CANCELLED	Settlement
Job requisition	DRAFT -> PENDING_APPROVAL -> APPROVED -> OPEN -> ON_HOLD/ CLOSED/CANCELLED	Recruitment
Application	ACTIVE pipeline stage -> OFFER/ REJECTED/WITHDRAWN -> HIRED	Recruitment
Offer	DRAFT -> INTERNAL_REVIEW -> ISSUED -> ACCEPTED/DECLINED/EXPIRED/ WITHDRAWN	Recruitment
AI learning item	NEW -> TRIAGED -> IN_REVIEW -> APPROVED/EDITED_APPROVED/ REJECTED -> PROMOTED	AI
AI action proposal	PROPOSED -> AWAITING_CONFIRMATION -> CONFIRMED -> EXECUTED/FAILED/ CANCELLED/EXPIRED; sensitive may insert APPROVAL_REQUIRED	AI

Appendix I - Background Jobs and Schedulers

Job	Owner	Trigger	Idempotency key / safety
Document expiry scan	Documents	Daily configured time	document_id + due-date window
Leave accrual	Leave	Scheduled monthly/daily by policy	employment + policy + accrual period
Attendance day calculation	Attendance	After events/import or schedule	employment + work_date + calc_version
Attendance period close candidate	Attendance	Period schedule	legal entity + period
Payroll input snapshot/load	Payroll	Explicit run action	run_id + employment_id + input version
Payslip generation	Payroll	After finalized/publish command	run_employee + payslip version
Connector outbox dispatch	Integrations	Continuous/interval	outbox_message_id
Connector retry	Integrations	Backoff schedule	message/sync id
Notification delivery	Notifications	Queue driven	message_id + channel
Report export	Reporting	User/schedule	export_job_id
AI knowledge embedding	AI	After approved document/version	knowledge_document_id + embedding_model_version
AI eval suite	AI	Release candidate/manual/scheduled	build + eval_suite_version + model/prompt version
AI semantic-cache cleanup	AI	Scheduled	cache entry version/expiry
Backup verification orchestration	Operations	Scheduled	backup set id

Background jobs must acquire an application-level idempotency key or database uniqueness guard. A job retry must not duplicate leave accrual, payroll input, external ERP transaction, notification, or AI action.

Appendix J - Frontend Architecture for Independent Module Teams

The frontend should mirror backend module boundaries enough to reduce ownership collisions. Shared UI primitives live in a design system; business feature folders own their routes, forms, query hooks, permission checks and localization keys.

```
web/src/  
  app/  
    router/  
    providers/  
    auth/  
  shared/  
    design-system/  
    api-client/  
    forms/  
    tables/  
    i18n/  
    errors/  
  features/  
    people/  
    attendance/  
    leave/  
    payroll/  
    recruitment/  
    reporting/  
    ai/
```

- Generate typed API clients from OpenAPI where practical; do not hand-copy DTOs across teams.
- Module screens must not call another module endpoint just to reconstruct data already exposed by an approved composite/read endpoint. Request a contract/read model instead.
- Permission checks in UI improve UX but backend remains authoritative.
- Arabic RTL and English LTR are first-class. Text is localization-key based, not hard-coded.
- Sensitive payroll/person fields use reusable masking/reveal components and explicit permission checks.
- Frontend module owners provide Storybook/component examples or equivalent for complex reusable components if adopted.

Appendix K - ESS / MSS Experience Contract

ESS/MSS is primarily a role-scoped experience across business modules, not a duplicate database schema. It should reuse the same domain APIs with tighter scopes and purpose-built composite endpoints where necessary.

Experience	Examples	Source modules
Employee Self-Service	My profile, documents, leave balance/request, attendance, payslips, AI help	People, Documents, Leave, Attendance, Payroll, AI
Manager Self-Service	Team list, pending leave, attendance exceptions, selected compensation/report visibility, requisition actions	People, Approvals, Leave, Attendance, Reporting, Recruitment
HR Workspace	Employee lifecycle, contracts, policies, payroll preparation, recruitment	Most modules
Finance/Payroll Workspace	Payroll review/finalization, payment/export, variances	Payroll, Reporting, Integrations

Do not create separate `ess_*` copies of employee/leave/payroll data. ESS/MSS is an authorization and UX projection over source modules.

Appendix L - Configuration and Entitlement Data Model

Concept	Suggested owner/table	Rule
Feature entitlement	platform.feature_entitlements	Commercial access; not authorization.
Tenant setting	platform.tenant_settings	Typed, documented keys; do not create an unbounded dumping ground.
Custom field definition	future configuration schema or module-owned extension definition	Only for sparse customer metadata; important domain fields become real schema.
Payroll discretionary component	payroll.payroll_components	Supported calculation types, not arbitrary executable code.
Approval policy	approvals.approval_policies	Start simple/sequential; expand only when real cases require.
Recruitment pipeline	recruitment.pipeline_versions/stages	Tenant configurable and versioned.
AI policy	ai.tenant_ai_settings	Mode, provider, egress, budget, allowed tool/ action classes.

Appendix M - Error Code and Observability Contract

Error code format:
<MODULE>_<RESOURCE>_<CONDITION>

Examples:
PEOPLE_EMPLOYMENT_NOT_FOUND
LEAVE_REQUEST_OVERLAPS_EXISTING
PAYROLL_RUN_ALREADY_FINALIZED
COMPLIANCE_RULE_VERSION_NOT_EFFECTIVE
RECRUITMENT_OFFER_NOT_ACCEPTED
AI_TOOL_NOT_AUTHORIZED
AI_POLICY_SOURCE_NOT_FOUND
AI_EXTERNAL_EGRESS_DISABLED

Errors exposed to users must be stable and safe. Debug stack traces stay server-side. Every background job and integration/AI provider call carries correlation ID and produces a supportable failure record.

Appendix N - Customer Deployment Configuration Checklist

A deployment is configured, not forked. The implementation team captures:

- Tenant/legal entities, branches, departments, positions and cost centers.
- Authentication mode: local, AD/LDAP/OIDC/Entra.
- Roles, permissions and data scopes.
- Employee numbering/import mapping and required documents.
- Attendance devices/import formats, shifts, schedules and approval period.
- Leave types/policies and company benefits above statutory baseline.
- Payroll components, pay groups, cutoffs, manual/import variables and output formats.
- Bank/accounting/ERP connectors.
- Recruitment pipeline, offer templates and public careers mode.
- AI mode: off/cloud/private, approved knowledge sources, external data-egress policy, budgets and allowed actions.
- Backup owner, restore procedure, RPO/RTO, update owner and support access method.

Appendix O - Recommended Team Assignment Examples

Module boundaries support different team sizes. Do not create artificial silos if only two programmers exist; one person can own multiple modules while preserving the boundaries.

Team size	Suggested ownership
2 developers	Dev A: Platform + People + Time; Dev B: Payroll + Recruitment + Reporting/AI, with shared frontend/integration review. External payroll SME.
3 developers	Dev A: Platform/People; Dev B: Time/Payroll/Compliance; Dev C: Recruitment/Frontend/Reporting/AI initially. Rotate integration ownership.
4 developers	Platform/People; Time/Leave; Payroll/Compliance; Recruitment/Frontend/AI, with explicit cross-review.
5 developers	Platform; Workforce Core; Time/Leave; Payroll/Compliance; Talent/AI/Integration + frontend responsibility split by actual skill set.

Regardless of size, one person must act as **integration/architecture owner** for contract changes. This is a responsibility, not necessarily a full-time architect role.

Appendix P - Deferred Target-State Modules

P.1 Performance Management Module

Status: architected as a future product module; not required to block Payroll/Recruitment/AI delivery.

Schema: performance

Purpose: manage performance cycles, goals/OKRs, competencies, reviews, calibration and acknowledged outcomes without coupling performance scoring to payroll calculations.

Owns

- PerformanceCycle
- Goal / GoalAssignment
- CompetencyFramework / Competency
- Review / ReviewSection / ReviewResponse
- CalibrationSession
- PerformanceOutcome

Does not own

- Employee or manager hierarchy; obtains as-of snapshots from People.
- Salary changes or bonuses; may publish an approved recommendation/input, but Payroll/People own actual compensation changes.
- Authentication or role assignment.

Primary tables

Table	Key relations	Purpose
performance.cycles	tenant/legal entity	Review window, eligibility and status.
performance.goals	cycle, owner employment	Goal definition, weight, target and progress.
performance.competency_frameworks	tenant	Versioned competency model.
performance.reviews	cycle, subject employment, reviewer employment	Review envelope and status.
performance.review_responses	review, section/competency/goal	Scored/text evidence.
performance.calibration_sessions	cycle	Controlled calibration metadata.
performance.outcomes	review/subject	Final acknowledged outcome and recommendation metadata.

Public contracts

- CreatePerformanceCycle
- AssignGoals
- SubmitSelfReview
- SubmitManagerReview

- CalibrateReview
- FinalizePerformanceOutcome
- GetPerformanceSummary

Events

- PerformanceCycleOpened
- PerformanceReviewSubmitted
- PerformanceOutcomeFinalized

Invariants

- Review history is versioned/auditable after submission.
- AI may summarize or assist with drafting, but must not silently invent evidence or make an irreversible employment decision.
- Compensation recommendations are not direct payroll writes.

P.2 Onboarding / Offboarding Orchestration

This should initially be an orchestration capability across People, Documents, Notifications, Approvals and optionally Assets rather than a duplicate employee lifecycle database. A future dedicated `lifecycle` schema is justified only if real customers require complex reusable task templates, dependencies and SLA tracking.

P.3 Asset Custody / Employee Relations

Asset custody, disciplinary cases, grievances and similar HR-administration capabilities should be added as bounded modules only when commercially required. Their records must link to `EmployeeId` / `EmploymentId`, maintain their own permissions/retention rules, and never be hidden inside free-form employee JSON.

Final Engineering Position

The platform should be built as a **standard reusable product with explicit module ownership**, not as a chain of customer-specific screens. The strongest long-term assets are the People/Employment model, historically reproducible Payroll, versioned Egypt Compliance, Recruitment-to-Hire boundary, integration architecture, and the governed AI Copilot/Learning Center. Multi-developer speed comes from stable contracts and ownership, not from letting every programmer access every table.